

Chapter 5. Collections

Most programs need to deal with multiple pieces of data. Your code might have to iterate through some transactions to calculate the balance of an account, for example, or display recent messages in a social media web application, or update the positions of characters in a game. In most kinds of applications, the ability to work with collections of information is likely to be important.

C# offers a simple kind of collection called an *array*. The CLR's type system supports arrays intrinsically, so they are efficient, but for some scenarios they can be too basic. The runtime libraries build on the fundamental services provided by arrays to provide more powerful and flexible collection types. I'll start with arrays, because they are the foundation of most of the collection classes.

Arrays

An array is an object that contains multiple *elements* of a particular type. Each element is a storage location similar to a field, but whereas with fields we give each storage slot a name, array elements are simply numbered. The number of elements is fixed for the lifetime of the array, so you must specify the size when you create it. [Example 5-1](#) shows the syntax for creating new arrays.

Example 5-1. Creating arrays

```
int[] numbers = new int[10];  
string[] strings = new string[numbers.Length];
```

As with all objects, we construct an array with the `new` keyword followed by a type name, but instead of parentheses with constructor arguments,

we put square brackets containing the array size. As the example shows, the expression defining the size can be a constant, but it doesn't have to be—the second array's size will be determined by evaluating `numbers.Length` at runtime. In this case, the second array will have 10 elements because we're using the first array's `Length` property. All arrays have this read-only property, and it returns the total number of elements in the array.

The `Length` property's type is `int`, which means it can cope with arrays of up to about 2.1 billion elements. In a 32-bit process, the limiting factor on array size is likely to be available address space, but in 64-bit processes, larger arrays are possible, so there's also a `LongLength` property of type `long`. However, you don't see that used much, because the runtime does not currently support creation of arrays with more than 2,147,483,591 (0x7FFFFFFC7) elements in any single dimension. So only rectangular multidimensional arrays (described later in this chapter) can contain more elements than `Length` can report. And even those have an upper limit of 4,294,967,295 (0xFFFFFFFF) elements on the current version of .NET.

In [Example 5-1](#), I've broken my normal rule of avoiding redundant type names in variable declarations. The initializer expressions make it clear that the variables are arrays of `int` and `string`, respectively, so I'd normally use `var` for this sort of code, but I've made an exception here so that I can show how to write the name of an array type. Array types are distinct types in their own right, and if we want to refer to the type that is a single dimensional array of some particular element type, we put `[]` after the element type name.

All array types derive from a common base class called `System.Array`. This defines the `Length` and `LongLength` properties and various other members we'll be looking at in due course. You can use array types in all the usual places you can use other types. So you could declare a field, or a method parameter of type `string[]`. You can also use an array type as a generic type argument. For example, `IEnumerable<int[]>` would be a sequence of arrays of integers (each of which could be a different size).

An array type is always a reference type, regardless of the element type. Nonetheless, the choice between reference type and value type elements makes a significant difference in an array's behavior. As discussed in [Chapter 3](#), when an object has a field with a value type, the value itself lives inside the memory allocated for the object. The same is true for arrays—when the elements are value types, the value lives in the array element itself, but with a reference type, elements contain only references. Each instance of a reference type has its own identity, and since multiple variables may all end up referring to that instance, the CLR needs to manage its lifetime independently of any other object, so it will end up with its own distinct block of memory. So while an array of 1,000 `int` values can all live in one contiguous memory block, with reference types, the array just contains the references, not the actual instances. An array of 1,000 different strings would need 1,001 heap blocks—one for the array and one for each string.

NOTE

When using reference type elements, you're not obliged to make every element in an array of references refer to a distinct object. You can leave as many elements as you like set to `null`, and you're also free to make multiple elements refer to the same object. This is just another variation on the theme that references in array elements work in much the same way as they do in local variables and fields.

To access an element in an array, we use square brackets containing the index of the element we'd like to use. The index is zero-based. [Example 5-2](#) shows a few examples.

Example 5-2. Accessing array elements

```
// Continued from Example 5-1
numbers[0] = 42;
numbers[1] = numbers.Length;
numbers[2] = numbers[0] + numbers[1];
numbers[numbers.Length - 1] = 99;
```

As with the array's size at construction, the array index can be a constant, but it can also be a more complex expression, calculated at runtime. In fact, that's also true of the part that comes directly before the opening bracket. In [Example 5-2](#), I've just used a variable name to refer to an array, but you can use brackets after any array-typed expression.

[Example 5-3](#) retrieves the first element of an array returned by a method call. (The details of the example aren't strictly relevant, but in case you're wondering, it finds the copyright message associated with the component that defines an object's type. For example, if you pass a `string` to the method, it will return "© Microsoft Corporation. All rights reserved." This uses the reflection API and custom attributes, the topics of [Chapters 13](#) and [14](#).)

Example 5-3. Convoluted array access

```
public static string GetCopyrightForType(object o)
{
    Assembly asm = o.GetType().Assembly;
    var copyrightAttribute = (AssemblyCopyrightAttribute)
        asm.GetCustomAttributes(typeof(AssemblyCopyrightAttribute), true)[0];
    return copyrightAttribute.Copyright;
}
```

Expressions involving array element access are special, in that C# considers them to be a kind of variable. This means that as with local variables and fields, you can use them on the lefthand side of an assignment statement, whether they're simple, like the expressions in [Example 5-2](#), or more complex, like those in [Example 5-3](#). You can also use them with the `ref` keyword (as described in [Chapter 3](#)) to pass a reference to a particular element to a method, to store it in a `ref` local variable, or to return it from a method with a `ref` return type.

The CLR always checks the index against the array size. If you try to use either a negative index or an index greater than or equal to the length of the array, the runtime will throw an `IndexOutOfRangeException`.

Although the size of an array is invariably fixed, its contents are always modifiable—there is no such thing as a read-only array. (As we'll see in

[“ReadOnlyCollection<T>”](#), .NET provides a class that can act as a read-only façade for an array.) You can, of course, create an array with an immutable element type, and this will prevent you from modifying the element in place. So [Example 5-4](#), which uses the immutable `Complex` value type provided by .NET, will not compile.

Example 5-4. How not to modify an array with immutable elements

```
var values = new Complex[10];  
// These lines both cause compiler errors:  
values[0].Real = 10;  
values[0].Imaginary = 1;
```

The compiler complains because the `Real` and `Imaginary` properties are read-only; `Complex` does not provide any way to modify its values. Nevertheless, you can modify the array: even if you can’t modify an existing element in place, you can always overwrite it by supplying a different value, as [Example 5-5](#) shows.

Example 5-5. Modifying an array with immutable elements

```
var values = new Complex[10];  
values[0] = new Complex(10, 1);
```

Read-only arrays wouldn’t be much use in any case, because all arrays start out filled with a default value that you don’t get to specify. The CLR fills the memory for a new array with zeros, so you’ll see `0`, `null`, or `false`, depending on the array’s element type.

WARNING

If you write a zero-argument constructor for a `struct`, you might have expected array creation to invoke constructors of this kind automatically. It does not.

For some applications, all-zero (or equivalent) content might be a useful initial state for an array, but in some cases, you’ll want to set some other content before starting to work.

Array Initialization

The most straightforward way to initialize an array is to assign values into each element in turn. [Example 5-6](#) creates a `string` array, and since `string` is a reference type, creating a five-element array doesn't create five strings. Our array starts out with five nulls. (This is true even if you've enabled C#'s nullable references feature, as described in [Chapter 3](#). Unfortunately, array initialization is one of the holes that make it impossible for that feature to offer absolute guarantees of non-nullness.) So the example goes on to populate each array element with a reference to a string.

Example 5-6. Laborious array initialization

```
var workingWeekDayNames = new string[5];
workingWeekDayNames[0] = "Monday";
workingWeekDayNames[1] = "Tuesday";
workingWeekDayNames[2] = "Wednesday";
workingWeekDayNames[3] = "Thursday";
workingWeekDayNames[4] = "Friday";
```

This works, but it is unnecessarily verbose. C# supports a shorter syntax that achieves the same thing, shown in [Example 5-7](#). The compiler turns this into code that works like [Example 5-6](#). With arrays of numbers (e.g., an `int[]`) the shorter syntax can generate more efficient code: instead of emitting a series of assignments, it can embed the array's values as a chunk of raw data in the compiled output, enabling it to initialize the entire array from that data with a single call to a .NET runtime library helper function. (The embedded data is read-only, and will be copied into a new array each time the code runs, so the effect is equivalent to sequence of assignments. It's just faster, and it results in more compact compiled code.)

Example 5-7. Initializing an array with a collection expression

```
string[] workingWeekDayNames =
    ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"];
```

The syntax in [Example 5-7](#) is called a *collection expression*, and it is new in C# 12.0. As you'll see later, you can also use it with other collection types. It is now the preferred way to initialize sequence-like collections because it is more compact than the older approaches, it supports a wider range of collection types, and the compiler strives to generate the most efficient initialization code possible for whichever collection type you have chosen. (If you upgrade an older project to .NET 8.0, you will typically see several messages from the compiler suggesting that you move over to this new syntax.)

Collection expressions can copy data from other collections using the *spread* syntax illustrated in [Example 5-8](#). This example creates a new array by copying the elements from the array in [Example 5-7](#) into a new array with additional elements at the start and end. Spread elements are not required to be of precisely the same type—although this example spreads one `string[]` into a new `string[]`, I could have incorporated any collection representing a sequence of `string` elements, such as the `List<string>` type described later in this chapter.

Example 5-8. Using a spread element to copy an array as part of a collection expression

```
string[] weekDayNames = ["Sunday", .. workingWeekDayNames, "Saturday"];
```

Collection expressions are designed to resemble the list patterns you saw in [Chapter 2](#), which is why spreads use the same `..` syntax as the slices that can appear in list patterns. However, while a list pattern can contain at most one slice, we're not limited to a single spread: a collection expression may contain any number of spread expressions.

Since collection expressions are new in C# 12.0, you are likely to come across existing code that uses the older *array initializer* syntax shown in [Example 5-9](#). That particular example is more verbose, because it explicitly states that we want a new array of type `string[]`. This means that there are still uses for this older syntax: if the compiler is unable to infer the required collection type, you can't use a collection expression because it's not clear whether you want an array, a `List<string>`, an `ImmutableList<string>`, or something else—all of those are potential target types

for the collection expression to the right of the `=` in [Example 5-7](#). That example avoids ambiguity by stating the type explicitly in the variable declaration, but if we had used `var`, the compiler would have complained that it couldn't determine which collection type we want. The more verbose initializer in [Example 5-9](#) avoids this.

Example 5-9. Array initializer syntax

```
var workingWeekDayNames = new string[]  
    { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" };
```

As it happens, C# has long supported one special case in which the older collection initializer syntax can omit the `new string[]` part. [Example 5-10](#) shows that if you specify the type explicitly in the variable declaration, you can write just the initializer list, leaving out the `new` keyword. This works only in variable initializer expressions, by the way; you can't use this syntax to create an array in other expressions, such as assignments or method arguments. (The newer collection expression syntax works in all those contexts, as does the more verbose initializer expression in [Example 5-9](#).) There's no practical difference between [Example 5-10](#) and the newer syntax in [Example 5-7](#), but since collection expressions offer additional flexibility in other contexts, and they are the most efficient way to initialize other collection types, they are likely to become the preferred idiomatic choice in these cases.

Example 5-10. Shorter array initializer syntax

```
string[] workingWeekDayNames =  
    { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" };
```

The array initializer syntax offers yet another variation: if all the expressions inside the array initializer list are of the same type, the compiler can infer the array type, so we can write just `new[]` without an explicit element type. [Example 5-11](#) does this.

Example 5-11. Array initializer syntax with element type inference

```
var workingWeekDayNames = new[]  
    { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" };
```

That was actually slightly longer than [Example 5-10](#). However, as with collection expressions and the other more verbose kinds of array initializers, this style is not limited to variable initialization. You can also use it when you need to pass an array as an argument to a method, for example. If the array you are creating will only be passed into a method and never referred to again, you may not want to declare a variable to refer to it. It might be neater to write the array directly in the argument list. [Example 5-12](#) passes an array of strings to a method using this technique.

Example 5-12. Array as argument

```
SetHeaders(new[] { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" });
```

In most cases, the new collection expression syntax is likely to be a better choice, but there are some circumstances where its flexibility works against it. If the `SetHeaders` method invoked by [Example 5-12](#) takes a first argument of type `string[]`, then a collection expression would be a more succinct choice. However, it would be different if its first argument's type was `IEnumerable<string>` (a type we'll be looking at shortly, which would indicate that `SetHeader` can work with many different collection types). In that case, using a collection expression causes the compiler to generate code that creates a read-only wrapper around an array. This ensures that if `SetHeaders` were to try casting its input to an `IList<T>` it would be prevented from modifying it. That would make it safe to reuse the same collection each time the code ran, although as it happens the current compiler doesn't seem to do that. (The C# language specification gives the compiler some latitude here, so there may be situations in which it will cache and reuse the collection.) In situations where this protection is unnecessary (e.g., because you control the code to which the collection is passed, and know it does not attempt any such modifications) just passing an array would be more efficient. To do that, you could use an array initializer as [Example 5-12](#) does, and not a collection expres-

sion, or you could cast a collection expression to an array type so that the compiler knows what type you require.

TIP

If you write code that initializes an array with a fixed set of contents, like some of the preceding examples, you should consider putting the array into a `static readonly` field. That way, the array is created just once and reused. Unless you really need a new array instance every time the code runs (e.g., because you go on to modify the array, or because the contents aren't the same every time), or the method in question will be invoked only once, it can be more efficient to reuse the same array object each time.

Searching and Sorting

Sometimes, you will not know the index of the array element you need. For example, suppose you are writing an application that shows a list of recently used files. Each time the user opens a file in your application, you would want to bring that file to the top of the list, and you'd need to detect when the file was already in the list to avoid having it appear multiple times. If the user happened to use your recent file list to open the file, you would already know it's in the list and at what offset. But what if the user opens the file some other way? In that case, you've got a file-name, and you need to find out where that appears in your list, if it's there at all.

Arrays can help you find the item you want in this kind of scenario. There are methods that examine each element in turn, stopping at the first match, and there are also methods that can work considerably faster if your array stores its elements in a particular order. To help with that, there are also methods for sorting the contents of an array into whichever order you require.

The static `Array.IndexOf` method provides the most straightforward way to search for an element. It does not need your array elements to be in any particular order: you just pass it the array in which to search and the value you're looking for, and it will walk through the elements until it finds a value equal to the one you want. It returns the index at which it

found the first matching element, or `-1` if it reached the end of the array without finding a match. [Example 5-13](#) shows how you might use this method as part of the logic for updating a list of recently opened files.

Example 5-13. Searching with `IndexOf`

```
int recentFileListIndex = Array.IndexOf(myRecentFiles, openedFile);
if (recentFileListIndex < 0)
{
    AddNewRecentEntry(openedFile);
}
else
{
    MoveExistingRecentEntryToTop(recentFileListIndex);
}
```

That example starts its search at the beginning of the array, but you have other options. The `IndexOf` method is overloaded, and you can pass an index from which to start searching and optionally a second number indicating how many elements you want it to look at before it gives up. There's also a `LastIndexOf` method, which works in reverse. If you do not specify an index, it starts from the end of the array and works backward. As with `IndexOf`, you can provide one or two more arguments, indicating the offset at which you'd like to start and the number of elements to check.

These methods are fine if you know precisely what value you're looking for, but often, you'll need to be a bit more flexible: you may want to find the first (or last) element that meets some particular criteria. For example, suppose you have an array representing the bin values for a histogram. It might be useful to find out which is the first nonempty bin. So rather than searching for a particular value, you'd want to find the first element with any value other than zero. [Example 5-14](#) shows how to use the `FindIndex` method to locate the first such entry.

Example 5-14. Searching with `FindIndex`

```
public static int GetIndexOfFirstNonEmptyBin(int[] bins)
    => Array.FindIndex(bins, IsNonZero);
```

```
private static bool IsNonZero(int value) => value != 0;
```

My `IsNonZero` method contains the logic that decides whether any particular element is a match, and I've passed that method as an argument to `FindIndex`. You can pass any method with a suitable signature—`FindIndex` requires a method that takes an instance of the array's element type and returns a `bool`. (Strictly speaking, it takes a `Predicate<T>`, which is a kind of delegate, something I'll discuss in [Chapter 9](#).) Since any method with a suitable signature will do, we can make our search criteria as simple or as complex as we like.

By the way, the logic for this particular example is so simple that writing a separate method for the condition is probably overkill. For simple cases such as these, you'd almost certainly use the lambda syntax (using `=>` to indicate that an expression represents an inline function) instead. That's also something I'll be discussing in [Chapter 9](#), so this is jumping ahead, but I'll just show how it looks because it's more concise. [Example 5-15](#) has exactly the same effect as [Example 5-14](#) but doesn't require us to declare and write a whole extra method explicitly.

Example 5-15. Using a lambda with `FindIndex`

```
public static int GetIndexOffirstNonEmptyBin(int[] bins)
    => Array.FindIndex(bins, value => value != 0);
```

As with `IndexOf`, `FindIndex` provides overloads that let you specify the offset at which to start searching and the number of elements to check before giving up. The `Array` class also provides `FindLastIndex`, which works backward—it corresponds to `LastIndexOf`, much as `FindIndex` corresponds to `IndexOf`.

When you're searching for an array entry that meets some particular criteria, you might not be all that interested in the index of the matching element—you might need to know only the value of the first match.

Obviously, it's pretty easy to get that: you can just use the value returned by `FindIndex` in conjunction with the array index syntax. However, you don't need to, because the `Array` class offers `Find` and `FindLast` meth-

ods that search in precisely the same way as `FindIndex` and `FindLastIndex` but that return the first or last matching value instead of returning the index at which that value was found.

An array could contain multiple items that meet your criteria, and you might want to find all of them. You could write a loop that calls `FindIndex`, adding one to the index of the previous match and using that as the starting point for the next search, repeating until either reaching the end of the array or getting a result of `-1`, indicating that no more matches were found. And that would be the way to go if you needed to know the index of each match. But if you are interested only in knowing all of the matching values, and do not need to know exactly where those values were in the array, you could use the `FindAll` method shown in [Example 5-16](#) to do all the work for you.

Example 5-16. Finding multiple items with `FindAll`

```
public static T[] GetNonNullItems<T>(T[] items) where T : class
    => Array.FindAll(items, value => value != null);
```

This takes any array with reference type elements and returns an array that contains only the non-null elements in that array.

All of the search methods I've shown so far run through an array's elements in order, testing each element in turn. This works well enough, but with large arrays it may be unnecessarily expensive, particularly in cases where comparisons are relatively complex. Even for simple comparisons, if you need to deal with arrays with millions of elements, this sort of search can take long enough to introduce visible delays. However, we can do much better. For example, given an array of values sorted into ascending order, a *binary search* can perform many orders of magnitude better. [Example 5-17](#) shows two methods. The first, `Sort`, sorts an array of numbers into ascending order. And if we have such a sorted array, we can then pass it to `Find`, which uses the `Array.BinarySearch` method.

Example 5-17. Sorting an array, and BinarySearch

```
void Sort(int[] numbers)
{
    Array.Sort(numbers);
}

int Find(int[] numbers, int searchFor)
{
    return Array.BinarySearch(numbers, searchFor);
}
```

Binary search is a widely used algorithm that exploits the fact that its input is sorted to be able to rule out half of the array at each step. It starts with the element in the middle of the array. If that happens to be the value required, it can stop, but otherwise, depending on whether the value it found is higher or lower than the value we want, it can know instantly which half of the array the value will be in (if it's present at all). It then leaps to the middle of the remaining half, and if that's not the right value, again it can determine which quarter will contain the target. At each step, it narrows the search down by half, and after halving the size a few times, it will be down to a single item. If that's not the value it's looking for, the item it wants is missing.

TIP

`BinarySearch` produces negative numbers when the value is not found. In these cases, this binary chop process will finish at the value nearest to the one we are looking for, and that might be useful information. So a negative number still tells us the search failed, but that number is the negation of the index of the closest match.

A binary search is more complex than a simple linear search, but with large arrays, it pays off because far fewer iterations are needed. Given an array of 100,000,000 elements, it has to perform only 27 steps instead of 100,000,000. Obviously, with smaller arrays, the improvement is reduced, and there will be some minimum size of array at which the relative complexity of a binary search outweighs the benefit. If your array contains

only 10 values, a linear search may well be faster. But a binary search is certainly the clear winner with 100,000,000 `int` elements. The cases that require the most work are where it finds no match (producing a negative result), and in these cases, `BinarySearch` determines that an element is missing around 20,000 times faster than the linear search performed by `Array.IndexOf` does. However, you need to take care: a binary search works only for data that is already ordered, and the cost of getting your data into order could well outweigh the benefits. With an array of 100,000,000 `int`s, you would need to do about 500 searches before the cost of sorting was outweighed by the improved search speed, and, of course, that would work only if nothing changed in the meantime that forced you to redo the sort. With performance tuning, it's always important to look at the whole scenario and not just the microbenchmarks.

Incidentally, `Array.BinarySearch` offers overloads for searching within some subsection of the array, similar to those we saw for the other search methods. It also lets you customize the comparison logic. This works with the comparison interfaces I showed in earlier chapters. By default, it will use the `Comparable<T>` implementation provided by the array elements themselves, but you can provide a custom `IComparer<T>` instead. The `Array.Sort` method I used to put the elements into order also supports narrowing down the range and using custom comparison logic.

There are other searching and sorting methods besides the ones provided by the `Array` class itself. All arrays implement `IEnumerable<T>` (where `T` is the array's element type), which means you can also use any of the operations provided by .NET's *LINQ to Objects* functionality. This offers a much wider range of features for searching, sorting, grouping, filtering, and generally working with collections of objects; [Chapter 10](#) will describe these features. Arrays have been in .NET for longer than LINQ, which is one reason for this overlap in functionality, but where arrays provide their own equivalents of standard LINQ operators, the array versions can sometimes be more efficient because LINQ is a more generalized solution.

Multidimensional Arrays

The arrays I've shown so far have all been one-dimensional, but C# supports two multidimensional forms: *jagged arrays* and *rectangular arrays*.

Jagged arrays

A jagged array is simply an array of arrays. The existence of this kind of array is a natural upshot of the fact that arrays have types that are distinct from their element type. Because `int[]` is a type, you can use that as the element type of another array. [Example 5-18](#) shows how to create an array of arrays of integers.

Example 5-18. Creating a jagged array with collection expressions

```
int[][] arrays =  
[  
    [1, 2],  
    [1, 2, 3, 4, 5, 6],  
    [1, 2, 4],  
    [1],  
    [1, 2, 3, 4, 5]  
];
```

I've used the collection expression syntax to initialize this array. As mentioned earlier, collection expressions are new in C# 12.0. [Example 5-19](#) shows how to create an identical array with the older array initialization syntax.

Example 5-19. Creating a jagged array with array initializers

```
var arrays = new int[5][]  
{  
    new[] { 1, 2 },  
    new[] { 1, 2, 3, 4, 5, 6 },  
    new[] { 1, 2, 4 },  
    new[] { 1 },  
    new[] { 1, 2, 3, 4, 5 }  
};
```


I've done something that wasn't strictly necessary here because I want to show a slightly odd feature of the jagged array constructor syntax. When using either collection expressions or array initializers, you don't have to specify the size explicitly, because the compiler will work it out for you. I've exploited that for the nested arrays in [Example 5-19](#), but I've set the size (5) explicitly for the outer array. I could have omitted it, but I wanted to show where the size appears, because it might not be where you would expect, particularly once you think about what the type name for an array really means.

In general, array types have the form `ElementType[]`, so if the element type is `int[]`, we'd expect the resulting array type to be written as `int[][]`, and that's what we see. The constructor syntax is a bit more peculiar. It declares an array of five arrays, and at a first glance, `new int[5][]` seems like a perfectly reasonable way to express that. It is consistent with array index syntax for jagged arrays; we can write `arrays[1][3]`, which fetches the second of those five arrays and then retrieves the fourth element from that second array. This is not a specialized syntax, by the way—there is no need for special handling here, because any expression that evaluates to an array can be followed by the index in square brackets. The expression `arrays[1]` evaluates to an `int[]` array, and so we can follow that with `[3]`.

However, the `new` keyword *does* treat jagged arrays specially. It makes them look consistent with array element access syntax, but it has to twist things a little to do that. With a one-dimensional array, the pattern for constructing a new array is `new ElementType[length]`, so for creating an array of five things, you'd expect to write `new ElementType[5]`. If the things you are creating are arrays of `int`, wouldn't you expect to see `int[]` in place of `ElementType`? That would imply that the syntax should be `new int[][5]`.

That would be logical, but it looks like it's the wrong way around, and that's because the array type syntax itself is effectively reversed. Arrays are constructed types, like generics. With generics, the name of the generic type from which we construct the actual type comes before the type argument (e.g., `List<int>` takes the generic `List<T>` type and constructs it with a type argument of `int`). If arrays had generic-like syntax,

we might expect to see `array<int>` for a one-dimensional array, `array<array<int>>` for two dimensions, and so on—the element type would come *after* the part that signifies that we want an array. But array types do it the other way around—the arrayness is signified by the `[]` characters, so the element type comes first. This is why the hypothetical logically correct syntax for array construction looks weird. C# avoids the weirdness by not getting overly stressed about logic here and just puts the size where most people expect it to go rather than where it arguably should go.

NOTE

The syntax extends in the obvious way—for example, `int[][][]` for the type and `new int[5][][]` for construction. C# does not define particular limits to the number of dimensions, but there are some implementation-specific runtime limits. (Microsoft's compiler didn't flinch when I asked for a 4,000-dimensional jagged array, but the CLR refused to load the resulting program. In fact, it wouldn't load anything with more than 2,816 dimensions.)

Examples [5-18](#) and [5-19](#) both initialize an array with five one-dimensional `int[]` arrays. The layout of the code should make it fairly clear why this sort of array is referred to as *jagged*: each row has a different length. With arrays of arrays, there is no requirement for a rectangular layout. I could go further. Arrays are reference types, so I could have set some rows to `null`. If I abandoned both the collection expression and array initializer syntaxes and initialized the array elements individually, I could have decided to make some of the one-dimensional `int[]` arrays appear in more than one row.

Because each row in this jagged array contains an array, I've ended up with six objects here—the five `int[]` arrays and then the `int[][]` array that contains references to them. If you introduce more dimensions, you'll get yet more arrays. For certain kinds of work, the nonrectangularity and the large numbers of objects can be problematic, which is why C# supports another kind of multidimensional array.

Rectangular arrays

A rectangular array is a single array object that supports multidimensional indexing. If C# didn't offer multidimensional arrays, we could build something a bit like them by convention. If you want an array with 10 rows and 5 columns, you could construct a one-dimensional array with 50 elements, and then use code like `myArray[i + (5 * j)]` to access it, where `i` is the column index and `j` is the row index. That would be an array that you had chosen to think of as being two-dimensional, even though it's really just one big contiguous block. A rectangular array is essentially the same idea, but where C# does the work for you. [Example 5-20](#) shows how to declare and construct rectangular arrays. (This is one scenario in which we can't use the new collection expression syntax, because that only knows how to create one-dimensional collections. It works for jagged arrays because those are one-dimensional arrays of one-dimensional arrays: each object constructed has just a single dimension. With rectangular arrays, we have a single multidimensional object.)

NOTE

Rectangular arrays are not just about convenience. There's a type safety aspect too: `int[,]` is a different type than `int[]` or `int[,]`, so if you write a method that expects a two-dimensional rectangular array, C# will not allow anything else to be passed.

Example 5-20. A rectangular array

```
int[,] grid = new int[5, 10];
var smallerGrid = new int[,]
{
    { 1, 2, 3, 4 },
    { 2, 3, 4, 5 },
    { 3, 4, 5, 6 }
};
```

Rectangular array type names use only a single pair of square brackets, no matter how many dimensions they have. The number of commas inside the brackets denotes the number of dimensions, so this example with

one comma is two-dimensional. The runtime seems to impose a much lower limit on the number of dimensions than for a jagged array. .NET 8.0 won't load a program that tries to use more than 32 dimensions in a rectangular array.

The multidimensional initializer syntax is very similar to the older initializer syntax for multidimensional arrays (see [Example 5-19](#)), except I do not start each row with `new[]`, because this is one big array, not an array of arrays. The numbers in [Example 5-20](#) form a shape that is clearly rectangular, and if you attempt to make things jagged (with different row sizes), the compiler will report an error. This extends to higher dimensions. If you wanted a three-dimensional “rectangular” array, it would need to be a *cuboid*. [Example 5-21](#) shows a cuboid array. You could think of the initializer as being a list of two rectangular slices making up the cuboid. And you can go higher, with *hypercuboid* arrays (although they are still known as rectangular arrays, regardless of how many dimensions you use).

Example 5-21. A $2 \times 3 \times 5$ cuboid “rectangular” array

```
var cuboid = new int[,,]
{
    {
        { 1, 2, 3, 4, 5 },
        { 2, 3, 4, 5, 6 },
        { 3, 4, 5, 6, 7 }
    },
    {
        { 2, 3, 4, 5, 6 },
        { 3, 4, 5, 6, 7 },
        { 4, 5, 6, 7, 8 }
    }
};
```

The syntax for accessing rectangular arrays is predictable enough. If the second variable from [Example 5-20](#) is in scope, we could write `smallerGrid[2, 3]` to access the final item in the array; as with single-dimensional arrays, indices are zero-based, so this refers to the third row's fourth item.

Remember that an array's `Length` property returns the total number of elements in the array. Since rectangular arrays have all the elements in a single array (rather than being arrays that refer to some other arrays), this will return the product of the sizes of all the dimensions. A rectangular array with 5 rows and 10 columns would have a `Length` of 50, for example. If you want to discover the size along a particular dimension at runtime, use the `GetLength` method, which takes a single `int` argument indicating the dimension for which you'd like to know the size.

Copying and Resizing

Sometimes you will want to move chunks of data around in arrays. You might want to insert an item in the middle of an array, moving the items that follow it up by one position (and losing one element at the end, since array sizes are fixed). Or you might want to move data from one array to another, perhaps one of a different size.

The static `Array.Copy` method takes two references to arrays, along with a number indicating how many elements to copy. It offers overloads so that you can specify the positions in the two arrays at which to start the copy. (The simpler overload starts at the first element of each array.) You are allowed to pass the same array as the source and destination, and it will handle overlap correctly: the copy acts as though the elements were first all copied to a temporary location before starting to write them to the target.

WARNING

As well as the static `Copy` method, the `Array` class defines a nonstatic `CopyTo` method, which copies the entire array into a target array, starting at the specified offset. This method is present because all arrays implement certain collection interfaces, including `ICollection<T>` (where `T` is the array's element type), which defines this `CopyTo` method. It is less flexible than `Copy` — `CopyTo` cannot copy a subrange of the array. In cases where either method would work, the documentation recommends using `Array.Copy` — `CopyTo` is just for the benefit of general-purpose code that can work with any implementation of a collection interface.

Copying elements from one array to another can become necessary when you need to deal with variable amounts of data. You would typically allocate an array larger than initially necessary, and if this eventually fills up, you'll need a new, larger array, and you'd need to copy the contents of the old array into the new one. In fact, the `Array` class can do this for you for one-dimensional arrays with its `Resize` method. The method name is slightly misleading, because arrays cannot be resized, so it allocates a new array and copies the data from the old one into it. `Resize` can build either a larger or a smaller array, and if you ask it for a smaller one, it will just copy as many elements as will fit.

While I'm talking about methods that copy the array's data around, I should mention `Reverse`, which simply reverses the order of the array's elements. Also, while this isn't strictly about copying, the `Array.Clear` method is often useful in scenarios where you're juggling array sizes—it allows you to reset some range of the array to its initial zero-like state.

These methods for moving data around within arrays are useful for building more flexible data structures on top of the basic services offered by arrays. But you often won't need to use them yourself, because the runtime libraries provide several useful collection classes that do this for you.

List<T>

The `List<T>` class, defined in the `System.Collections.Generic` namespace, contains a variable-length sequence of elements of type `T`. It provides an indexer that lets you get and set elements by number, so a `List<T>` behaves like a resizable array. It's not completely interchangeable—you cannot pass a `List<T>` as the argument for a parameter that expects a `T[]` array—but both arrays and `List<T>` implement various common generic collection interfaces that we'll be looking at later. For example, if you write a method that accepts an `ICollection<T>`, it will be able to work with either an array or a `List<T>`.

Although code that uses an indexer resembles array element access, it is not quite the same thing. An indexer is a kind of property, so it has the

same issues with mutable value types that I discussed in [Chapter 3](#). Given a variable `pointList` of type `List<Point>` (where `Point` is the mutable value type in the `System.Windows` namespace), you cannot write `pointList[2].X = 2`, because `pointList[2]` returns a copy of the value, and this code is effectively asking to modify that temporary copy. This would lose the update, so C# forbids it. But this does work with arrays. If `pointArray` is of type `Point[]`, `pointArray[2]` does not *get* an element, it *identifies* an element, making it possible to modify an array element's value in situ by writing `pointArray[2].X = 2`. (Since `ref` return values were added to C#, it has become possible to write indexers that work this way, but `List<T>` and `ICollection<T>` were created long before that.) With immutable value types such as `Complex`, this distinction is moot, because you cannot modify their values in place in any case—you would have to overwrite an element with a new value whether using an array or a list.

Unlike an array, a `List<T>` provides methods that change its size. The `Add` method appends a new element to the end of the list, while `AddRange` can add several. `Insert` and `InsertRange` add elements at any point in the list, shuffling all the elements after the insertion point down to make space. These four methods all make the list longer, but `List<T>` also provides `Remove`, which removes the first instance of the specified value; `RemoveAt`, which removes an element at a particular index; and `RemoveRange`, which removes multiple elements starting at a particular index. These all shuffle elements back down, closing up the gap left by the removed element or elements, making the list shorter.

NOTE

`List<T>` uses an array internally to store its elements. This means all the elements live in a single block of memory, and it stores them contiguously. This makes normal element access very efficient, but it is also why insertion needs to shift elements up to make space for the new element, and removal needs to shift them down to close up the gap.

[Example 5-22](#) shows one way to create a `List<T>`. It's just a class, so we can use the normal constructor syntax. It shows how to add and remove

entries and also how to access elements using the array-like indexer syntax. This also shows that `List<T>` provides its size through a `Count` property. This name may seem needlessly different than the `Length` provided by arrays, but there is a reason: this property is defined by `ICollection<T>`, which `List<T>` implements. Not all `ICollection<T>` implementations are sequences, so `Length` would in some cases be a misnomer. (As it happens, arrays also offer `Count`, because they also implement `ICollection` and `ICollection<T>`. However, they use explicit interface implementation, meaning that you can see an array's `Count` property only through a reference of one of these interface types.)

Example 5-22. Using a `List<T>`

```
var numbers = new List<int>();
numbers.Add(123);
numbers.Add(99);
numbers.Add(42);
Console.WriteLine(numbers.Count);
Console.WriteLine($"{numbers[0]}, {numbers[1]}, {numbers[2]}");

numbers[1] += 1;
Console.WriteLine(numbers[1]);

numbers.RemoveAt(1);
Console.WriteLine(numbers.Count);
Console.WriteLine($"{numbers[0]}, {numbers[1]}");
```

Because a `List<T>` can grow and shrink as required, you don't need to specify its size at construction. However, if you want to, you can specify its *capacity*. A list's capacity is the amount of space it currently has available for storing elements, and this will often be larger than the number of elements it contains. To avoid allocating a new internal array every time you add or remove an element, it keeps track of how many elements are in use independently of the size of the array. When it needs more space, it will overallocate, creating a new array that is larger than needed by an amount proportional to the size. This means that, if your program repeatedly adds items to a list, the larger it gets, the less frequently it needs to allocate a new array, but the proportion of spare capacity after each reallocation will remain about the same.

If you know up front that you will eventually store a specific number of elements in a list, you can pass that number to the constructor, and it will allocate exactly that much capacity, meaning that no further reallocation will be required. If you get this wrong, it won't cause an error—you're just requesting an initial capacity, and it's OK to change your mind later.

If the idea of unused memory going to waste in a list offends you, but you don't know exactly how much space will be required before you start, you could call the `TrimExcess` method once you know the list is complete. This reallocates the internal storage to be exactly large enough to hold the list's current contents, eliminating waste. This will not always be a win. To ensure that it is using exactly the right amount of space, `TrimExcess` has to create a new array of the right size, leaving the old, oversized one to be reclaimed by the garbage collector later on, and in some scenarios, the overhead of forcing an extra allocation just to trim things down to size may be higher than the overhead of having some unused capacity.

Lists have a third constructor. Besides the default constructor, and the one that takes a capacity, you can also pass in a collection of data with which to initialize the list. You can pass any `IEnumerable<T>`.

You can provide initial content for lists in the same way as with an array, by writing a collection expression. [Example 5-23](#) loads the same three values into a new list as at the start of [Example 5-22](#).

Example 5-23. Initializing a list with a collection expression

```
List<int> numbers = [123, 99, 42];
```

As with arrays, the collection expression syntax was introduced in C# 12.0, and there is an older, slightly more verbose syntax. [Example 5-24](#) has the same effect as [Example 5-23](#).

Example 5-24. List initializer

```
var numbers = new List<int> { 123, 99, 42 };
```

If you're not using `var`, you can omit the type name after the `new` keyword, as [Example 5-25](#) shows. But in contrast to arrays, you cannot omit the `new` keyword entirely. I've used a target-typed `new()` expression here to avoid writing out the type name a second time.

Example 5-25. List initializer with target-typed `new`

```
List<int> numbers = new() { 123, 99, 42 };
```

Examples [5-24](#) and [5-25](#) are equivalent, and each compile into code that calls `Add` once for each item in the list. You can use this syntax with any type that has a suitable `Add` method and implements the `IEnumerable` interface. This works even if `Add` is an extension method. (So if some type implements `IEnumerable`, but does not supply an `Add` method, you are free to use this initializer syntax if you provide your own `Add`.)

[Example 5-23](#) generates marginally more efficient code. The collection expression syntax not only recognizes the same pattern as the initializer syntax (an `IEnumerable` implementation and an `Add` method), but it also has some additional tricks. If the collection type offers a constructor with a single argument of type `int` named `capacity`, it will use that to construct the collection, passing in the number of elements that will be in the list once initialization is complete. `List<T>` has such a constructor, and this enables it to allocate exactly the right amount of memory to hold its values. Examples [5-24](#) and [5-25](#) won't do this, and will just invoke the zero-arguments constructor, forcing `List<T>` to guess how many elements it will be asked to hold. The current implementation allocates enough space to hold four elements, which is slightly more than this example requires. In cases that end up with more than four elements, `List<T>` ends up discarding the original four element array it created and replacing it with a larger one (twice as large as before in the current implementation). It would be possible to avoid this mismatch of size and capacity in the initializer syntax examples by passing an argument of `3` to the constructor, but in [Example 5-23](#) the compiler does that for us automatically. (The compiler can also generate code that efficiently loads all of the data into the list at once, instead of generating multiple calls to `Add`.) If we've used the spread syntax to copy elements from one or more other

collections, it will attempt to determine the total number of elements before constructing the `List<T>`. It is possible to create situations where it can't do this (because not all collection types know in advance how many elements they will eventually produce), but in cases where it's possible to do so, the C# compiler will generate code that optimizes memory allocation when you use the collection expression syntax. The older initializer syntax does not do this.

`List<T>` provides `IndexOf`, `LastIndexOf`, `Find`, `FindLast`, `FindAll`, `Sort`, and `BinarySearch` methods for finding and sorting list elements. These provide the same services as their array namesakes, although `List<T>` chooses to provide these as instance methods rather than statics.

We've now seen two ways to represent a list of values: arrays and lists. Fortunately, interfaces make it possible to write code that can work with either, so you won't need to write two sets of functions if you want to support both lists and arrays.

List and Sequence Interfaces

The runtime libraries define several interfaces representing collections. Three of these are relevant to simple linear sequences of the kind you can store in an array or a list: `ICollection<T>`, `IEnumerable<T>`, and `IList<T>`, all defined in the `System.Collections.Generic` namespace. There are three interfaces because different code makes different demands. Some methods need random access to any numbered element in a collection, but not everything does, and not all collections can support that—some sequences produce elements gradually, and there may be no way to leap straight to the *n*th element. Consider a sequence representing keypresses, for example—each item will emerge only as the user presses the next key. Your code can work with a wider range of sources if you opt for less demanding interfaces.

`IEnumerable<T>` is the most general of the collection interfaces because it demands the least from its implementers. I've mentioned it a few times already because it's an important interface that crops up a lot, but I've not

shown the definition until now. As [Example 5-26](#) shows, it declares just a single method.

Example 5-26. `IEnumerable<T>` and `IEnumerable`

```
public interface IEnumerable<out T> : IEnumerable
{
    IEnumerator<T> GetEnumerator();
}

public interface IEnumerable
{
    IEnumerator GetEnumerator();
}
```

Using inheritance, `IEnumerable<T>` requires its implementers also to implement `IEnumerable`, which appears to be almost identical. It's a nongeneric version of `IEnumerable<T>`, and its `GetEnumerator` method will typically do nothing more than invoke the generic implementation. The reason we have both forms is that the nongeneric `IEnumerable` has been around since .NET 1.0, which didn't support generics. The arrival of generics in .NET 2.0 made it possible to express the intent behind `IEnumerable` more precisely, but the old interface had to remain for compatibility. So these two interfaces effectively require the same thing: a method that returns an enumerator. What's an enumerator? [Example 5-27](#) shows both the generic and nongeneric interfaces.

Example 5-27. `IEnumerator<T>` and `IEnumerator`

```
public interface IEnumerator<out T> : IDisposable, IEnumerator
{
    T Current { get; }
}

public interface IEnumerator
{
    bool MoveNext();
    object Current { get; }
}
```

```
void Reset();  
}
```

The usage model for an `IEnumerable<T>` (and also `IEnumerable`) is that you call `GetEnumerator` to obtain an enumerator, which can be used to iterate through all the items in the collection. You call the enumerator's `MoveNext()`; if it returns `false`, it means the collection was empty. Otherwise, the `Current` property will now provide the first item from the collection. Then you call `MoveNext()` again to move to the next item, and for as long as it keeps returning `true`, the next item will be available in `Current`. (The `Reset` method is a historical artifact added to help compatibility with COM, the Windows pre-.NET cross-language object model. The documentation allows implementations to throw a `NotSupportedException` from `Reset`, so you will not normally use this method.)

NOTE

Notice that `IEnumerator<T>` implementations are required to implement `IDisposable`. You must call `Dispose` on enumerators once you're finished with them, because many of them rely on this.

The `foreach` loop in C# does all of the work required to iterate through an enumerable collection for you,¹ including generating code that calls `Dispose` even if the loop terminates early due to a `break` statement, an error, or, perish the thought, a `goto` statement. [Chapter 7](#) will describe the uses of `IDisposable` in more detail.

`IEnumerable<T>` is at the heart of LINQ to Objects, which I'll discuss in [Chapter 10](#). LINQ operators are available on any object that implements this interface. The runtime libraries define a related interface, `IAsyncEnumerable<T>`. Conceptually, this is identical to `IEnumerable<T>`: it represents the ability to provide a sequence of items. The difference is that it enables items to be enumerated asynchronously. As [Example 5-28](#) shows, this interface and its counterpart, `IAsyncEnumerator<T>`, resemble `IEnumerable<T>` and `IEnumerator<T>`. The main difference is the use of the asynchronous programming features `ValueTask<T>` and

`CancellationToken`, which [Chapter 16](#) will describe. There are also some minor differences: there are no nongeneric versions of these interfaces, and also, there's no facility to reset an existing asynchronous enumerator (although as noted earlier, many synchronous enumerators throw a `NotSupportedException` if you call `Reset`).

Example 5-28. `IAsyncEnumerable<T>` and `IAsyncEnumerator<T>`

```
public interface IAsyncEnumerable<out T>
{
    IAsyncEnumerator<T> GetAsyncEnumerator(
        CancellationToken cancellationToken = default);
}

public interface IAsyncEnumerator<out T> : IAsyncDisposable
{
    T Current { get; }

    ValueTask<bool> MoveNextAsync();
}
```

You can consume an `IAsyncEnumerable<T>` with a specialized form of `foreach` loop, in which you prefix it with the `await` keyword. This can only be used in a method marked with the `async` keyword. [Chapter 17](#) describes the `async` and `await` keywords in detail, and also the use of `await foreach`.

Although `IEnumerable<T>` is important and widely used, it's pretty restrictive. You can ask it only for one item after another, and it will hand them out in whatever order it sees fit. It does not provide a way to modify the collection, or even to find out how many items the collection contains without having to iterate through the whole lot. For these jobs, we have `ICollection<T>`, which is shown in [Example 5-29](#).

Example 5-29. `ICollection<T>`

```
public interface ICollection<T> : IEnumerable<T>, IEnumerable
{
    void Add(T item);
```

```

    void Clear();
    bool Contains(T item);
    void CopyTo(T[] array, int arrayIndex);
    bool Remove(T item);

    int Count { get; }
    bool IsReadOnly { get; }
}

```

This requires implementers also to provide `IEnumerable<T>`, but notice that this does not inherit the nongeneric `ICollection`. There is such an interface, but it represents a different abstraction: it's missing all of the methods except `CopyTo`. When introducing generics, Microsoft reviewed how the nongeneric collection types were used and concluded that the one extra method that the old `ICollection` added didn't make it noticeably more useful than `IEnumerable`. Worse, it also included a property called `SyncRoot` that was intended to help manage certain multithreaded scenarios but that turned out to be a poor solution to that problem in practice. So the abstraction represented by `ICollection` did not get a generic equivalent and has not been greatly missed. During the review, Microsoft also found that the absence of a general-purpose interface for modifiable collections was a problem, and so it made `ICollection<T>` fit that bill. It was not entirely helpful to attach this old name to a different abstraction, but since almost nobody was using the old nongeneric `ICollection`, it doesn't seem to have caused much trouble.

The third interface for sequential collections is `IList<T>`, and all types that implement this are required to implement `ICollection<T>`, and therefore also `IEnumerable<T>`. As you'd expect, `List<T>` implements `IList<T>`. Arrays implement it too, using their element type as the argument for `T`. [Example 5-30](#) shows how the interface looks.

Example 5-30. `IList<T>`

```

public interface IList<T> : ICollection<T>, IEnumerable<T>, IEnumerable
{
    int IndexOf(T item);
    void Insert(int index, T item);
    void RemoveAt(int index);
}

```



```
T this[int index] { get; set; }  
}
```

Again, although there is a nongeneric `IList`, this interface has no direct relationship to it, even though both interfaces represent similar concepts—the nongeneric `IList` has equivalents to the `IList<T>` members, and it also includes equivalents to most of `ICollection<T>`, including all the members missing from `ICollection`. So it would have been possible to require `IList<T>` implementations to implement `IList`, but that would have forced implementations to provide two versions of most members, one working in terms of the type parameter `T` and the other using `object`, because that's what the old nongeneric interfaces had to use. It would also force collections to provide the nonuseful `SyncRoot` property. The benefits would not outweigh these inconveniences, and so `IList<T>` implementations are not obliged to implement `IList`. They can if they want to, and `List<T>` does, but it's up to the individual collection class to choose.

One unfortunate upshot of the way these three generic interfaces are related is that they do not provide an abstraction representing indexed collections that are read-only, or even ones that are fixed-size. While `IEnumerable<T>` is a read-only abstraction, it's an in-order one with no way to go directly to the *n*th value. `IList<T>` provides indexed access, but it also defines methods for insertion and indexed removal, as well as mandating an implementation of `ICollection<T>` with its addition and value-based removal methods. You might be wondering how arrays can implement these interfaces, given that all arrays are fixed-size.

Arrays mitigate this problem by using explicit interface implementation to hide the `IList<T>` methods that can change a list's length, discouraging you from trying to use them. (As you saw in [Chapter 3](#), this technique enables you to provide a full implementation of an interface but to be selective about which members are directly visible.) However, you can store a reference to an array in a variable of type `IList<T>`, making those methods visible—[Example 5-31](#) uses this to call an array's `IList<T>.Add` method. However, this results in a runtime error.

Example 5-31. Trying (and failing) to enlarge an array

```
IList<int> array = new[] { 1, 2, 3 };  
array.Add(4); // Will throw an exception
```

The `Add` method throws a `NotSupportedException`, with an error message stating that the collection has a fixed size. If you inspect the documentation for `IList<T>` and `ICollection<T>`, you'll see that all the members that would modify the collection are allowed to throw this error. You can discover at runtime whether this will happen for *all* modifications with the `ICollection<T>` interface's `IsReadOnly` property. However, that won't help you discover up front when a collection allows only certain changes. (For example, an array's size is fixed, but you can still modify elements.)

This causes an irritating problem: if you're writing code that does in fact require a modifiable collection, there's no way to advertise that fact. If a method takes an `IList<T>`, it's hard to know whether that method will attempt to resize that list or not. Mismatches cause runtime exceptions, and those exceptions may well appear in code that isn't doing anything wrong, and where the mistake—passing the wrong sort of collection—was made by the caller. These problems are not showstoppers; in dynamically typed languages, this degree of compile-time uncertainty is in fact the norm, and it doesn't stop you from writing good code. It just shows that we can't rely on the type system to detect all mistakes.

There is a `ReadOnlyCollection<T>` class, but as we'll see later, that solves a different problem—it's a wrapper class, not an interface, so there are plenty of things that are fixed-size collections that do not present a `ReadOnlyCollection<T>`. If you were to write a method with a parameter of type `ReadOnlyCollection<T>`, it would not be able to work directly with certain kinds of collections (including arrays). In any case, it's not even the same abstraction—read-only is a tighter restriction than fixed-size.

.NET defines `IReadOnlyList<T>`, an interface that provides a better solution for representing read-only indexed collections (although it still doesn't help with modifiable fixed-size ones such as arrays). Like

`ICollection<T>`, it requires an implementation of `IEnumerable<T>`, but it does not require `ICollection<T>`. It defines two members: `Count`, which returns the size of the collection (just like `ICollection<T>.Count`), and a read-only indexer. This solves most of the problems associated with using `ICollection<T>` for read-only collections. One minor problem is that because it's newer than most of the other interfaces I've described here it is not universally supported. (It was introduced in .NET Framework 4.5 in 2012, seven years after `ICollection<T>`.) So if you come across an API that requires an `ICollection<T>`, you can be sure it will not attempt to modify the collection, but if an API requires `ICollection<T>`, it's difficult to know whether that's because it intends to modify the collection or merely because it was written before `ICollection<T>` was invented.

NOTE

Collections do not need to be read-only to implement `ICollection<T>`—a modifiable list can easily present a read-only façade. So this interface is implemented by all arrays and also `List<T>`.

The issues and interfaces I've just discussed raise a question: When writing code or classes that work with collections, what type should you use? With private implementation details (e.g., private fields, or inside methods) you will typically specify concrete types, not interfaces, because you don't need the flexibility to deal with different kinds of collections. But when it comes to the public-facing parts of your code (e.g., properties or method arguments) you will typically get the most flexibility if your API demands the least specific type it can work with. For example, if an `ICollection<T>` suits your needs, don't demand an `ICollection<T>`. Likewise, interfaces are usually better than concrete types, so you should prefer `ICollection<T>` over either `List<T>` or `T[]`. Just occasionally, there may be performance arguments for using a more specific type; if you have a tight loop critical to the overall performance of your application that works through the contents of a collection, you may find such code runs faster if it works only with array types, because the CLR may be able to perform better optimizations when it knows exactly what to expect. But in many cases, the difference will be too small to measure and will not justify the inconvenience of being tied to a particular implementation, so you should

never take such a step without measuring the performance for the task at hand to see what the benefit might be. (And if you're considering such a performance-oriented change, you should also look at the techniques described in [Chapter 18](#).) If you find that there is a possible performance win, but you're writing a shared library in which you want to provide both flexibility and the best possible performance, there are a couple of options for having it both ways. You could offer overloads so callers can pass in either an interface or a specific type. Alternatively, you could write a single public method that accepts the interface but that tests for known types and chooses between different internal code paths based on what the caller passes.

The interfaces we've just examined are not the only generic collection interfaces, because simple linear lists are not the only kind of collection. But before moving on to the others, I want to show enumerables and lists from the flip side: How do we implement these interfaces?

Implementing Lists and Sequences

It is often useful to provide information in the form of either an `IEnumerable<T>` or an `IList<T>`. The former is particularly important because .NET provides a powerful toolkit for working with sequences in the form of LINQ to Objects, which I'll show in [Chapter 10](#). LINQ to Objects provides various operators that all work in terms of `IEnumerable<T>`. `IList<T>` is a useful abstraction anywhere that random access to any element by index is required. Some frameworks expect an `IList<T>`. If you want to bind a collection of objects to some kind of list control, for example, some UI frameworks will expect either an `IList` or an `IList<T>`.

You could implement these interfaces by hand, as none of them is particularly complicated. However, C# and the runtime libraries can help. There is direct language-level support for implementing `IEnumerable<T>`, and the runtime libraries provide support for the generic and nongeneric list interfaces.

Implementing IEnumerable<T> with Iterators

C# supports a special form of method called an *iterator*. An iterator is a method that produces enumerable sequences using the `yield` keyword.

[Example 5-32](#) shows a simple iterator and some code that uses it. This will display numbers counting down from 5 to 1.

Example 5-32. A simple iterator

```
static IEnumerable<int> Countdown(int start, int end)
{
    for (int i = start; i >= end; --i)
    {
        yield return i;
    }
}

foreach (int i in Countdown(5, 1))
{
    Console.WriteLine(i);
}
```

An iterator looks much like any normal method, but the way it returns values is different. The iterator in [Example 5-32](#) has a return type of `IEnumerable<int>`, and yet it does not appear to return anything of that type. Instead of a normal `return` statement, it uses a `yield return` statement, and that returns a single `int`, not a collection. Iterators produce values one at a time with `yield return` statements, and unlike with a normal `return`, the method can continue to execute after returning a value—it's only when the method either runs to the end or decides to stop early with a `yield break` statement or by throwing an exception that it is complete. [Example 5-33](#) shows this rather more starkly. Each `yield return` causes a value to be emitted from the sequence, so this one will produce the numbers 1–3.

Example 5-33. A very simple iterator

```
public static IEnumerable<int> ThreeNumbers()
{
```

```

    yield return 1;
    yield return 2;
    yield return 3;
}

```

Although this is fairly straightforward in concept, the way it works is somewhat involved because code in iterators does not run in the same way as other code. Remember, with `IEnumerable<T>`, the caller is in charge of when the next value is retrieved; a `foreach` loop will get an enumerator and then repeatedly call `MoveNext()` until that returns `false`, and expect the `Current` property to provide the current value. So how do Examples [5-32](#) and [5-33](#) fit into that model? You might think that perhaps C# stores all the values an iterator yields in a `List<T>`, returning that once the iterator is complete, but it's easy to demonstrate that that's not true by writing an iterator that never finishes, such as the one in [Example 5-34](#).

Example 5-34. An infinite iterator

```

public static IEnumerable<BigInteger> Fibonacci()
{
    BigInteger v1 = 1;
    BigInteger v2 = 1;

    while (true)
    {
        yield return v1;
        BigInteger tmp = v2;
        v2 = v1 + v2;
        v1 = tmp;
    }
}

```

This iterator runs indefinitely; it has a `while` loop with a `true` condition, and it contains no `break` statement, so this will never voluntarily stop. If C# tried to run an iterator to completion before returning anything, it would get stuck here. (The numbers grow, so if it ran for long enough, the method would eventually terminate by throwing an `OutOfMemoryException`.) But if you try this, you'll find it starts returning values from the Fibonacci series immediately and will continue to do so

for as long as you continue to iterate through its output. Clearly, C# is not simply running the whole method before returning.

C# performs some serious surgery on your code to make this work. If you examine the compiler's output for an iterator using a tool such as ILDASM (the disassembler for .NET code, provided with the .NET SDK), you'll find it generates a private nested class that acts as the implementation for both the `IEnumerable<T>` that the method returns and also the `IEnumerator<T>` that the `IEnumerable<T>`'s `GetEnumerator` method returns. The code from your iterator method ends up inside this class's `MoveNext` method, but it is barely recognizable, because the compiler splits it up in a way that enables each `yield return` to return to the caller, but for execution to continue from where it left off the next time `MoveNext` is called. Where necessary, it will store local variables inside this generated class so that their values can be preserved across multiple calls to `MoveNext`. Perhaps the easiest way to get a feel for what C# has to do when compiling an iterator is to write the equivalent code by hand.

[Example 5-35](#) provides the same Fibonacci sequence as [Example 5-34](#) without the aid of an iterator. It's not precisely what the compiler does, but it illustrates some of the challenges.

Example 5-35. Implementing `IEnumerable<T>` by hand

```
public class FibonacciEnumerable :
    IEnumerable<BigInteger>, IEnumerator<BigInteger>
{
    private BigInteger v1;
    private BigInteger v2;
    private bool first = true;

    public BigInteger Current => v1;

    public void Dispose() { }

    object IEnumerator.Current => Current;

    public bool MoveNext()
    {
        if (first)
        {
```



```

        v1 = 1;
        v2 = 1;
        first = false;
    }
    else
    {
        BigInteger tmp = v2;
        v2 = v1 + v2;
        v1 = tmp;
    }

    return true;
}

public void Reset()
{
    first = true;
}

public IEnumerator<BigInteger> GetEnumerator() =>
    new FibonacciEnumerable();

IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

```

This is not a particularly complex example, because its enumerator is essentially in either of two states—either it is running for the first time and therefore needs to run the code that comes before the loop or it is inside the loop. Even so, this code is much harder to read than [Example 5-34](#), because the mechanics of supporting enumeration have obscured the essential logic.

The code would get even more convoluted if we needed to deal with exceptions. You can write `using` blocks and `finally` blocks, which enable your code to behave correctly in the face of errors, as I'll show in Chapters [7](#) and [8](#), and the compiler can end up doing a lot of work to preserve the correct semantics for these when the method's execution is split up over multiple iterations.² You wouldn't need to write too many enumerations by hand this way before being grateful that C# can do it for you.

Iterator methods don't have to return an `IEnumerable<T>`, by the way. If you prefer, you can return an `IEnumerator<T>` instead. And, as you saw earlier, objects that implement either of these interfaces also always implement the nongeneric equivalents, so if you need a plain `IEnumerable` or `IEnumerator`, you don't need to do extra work—you can pass an `IEnumerable<T>` to anything that was expecting a plain `IEnumerable`, and likewise for enumerators. If for some reason you want to provide one of these nongeneric interfaces and you don't wish to provide the generic version, you are allowed to write iterators that return the nongeneric forms directly.

One thing to be careful of with iterators is that they run very little code until the first time the caller calls `MoveNext`. So if you were to single-step through code that calls the `Fibonacci` method in [Example 5-34](#), the method call would appear not to do anything at all. If you try to step into the method at the point at which it's invoked, none of the code in the method runs. It's only when iteration begins that you'd see your iterator's body execute. This has a couple of consequences.

The first thing to bear in mind is that if your iterator method takes arguments, and you want to validate those arguments, you may need to do some extra work. By default, the validation won't happen until iteration begins, so errors will occur later than you might expect. If you want to validate arguments immediately, you will need to write a wrapper.

[Example 5-36](#) shows an example—it provides a normal method called `Fibonacci` that doesn't use `yield return` and will therefore not get the special compiler behavior for iterators. This normal method validates its argument before going on to call a nested iterator method. (This also illustrates that local methods can use `yield return`.)

Example 5-36. Iterator argument validation

```
public static IEnumerable<BigInteger> Fibonacci(int count)
{
    ArgumentOutOfRangeException.ThrowIfNegative(count);
    return Core(count);

    static IEnumerable<BigInteger> Core(int count)
    {
```

```

        BigInteger v1 = 1;
        BigInteger v2 = 1;

        for (int i = 0; i < count; ++i)
        {
            yield return v1;
            BigInteger tmp = v2;
            v2 = v1 + v2;
            v1 = tmp;
        }
    }
}

```

The second thing to remember is that iterators may execute several times. `IEnumerable<T>` provides a `GetEnumerator` that can be called many times over, and your iterator body will run from the start each time. So even though your iterator method may only have been called once, it could run multiple times.

Collection<T>

If you look at types in the runtime libraries, you'll find that when they offer properties that expose an implementation of `IList<T>`, they often do so indirectly. Instead of an interface, properties often provide some concrete type, although it's usually not `List<T>` either. `List<T>` is designed to be used as an implementation detail of your code, and if you expose it directly, you may be giving users of your class too much control. Do you want them to be able to modify the list? And even if you do, mightn't your code need to know when that happens?

The runtime libraries provide a `Collection<T>` class that is designed to be used as the base class for collections that a type will make publicly available. It is similar to `List<T>`, but there are two significant differences. First, it has a smaller API—it offers `IndexOf`, but all the other searching and sorting methods available for `List<T>` are missing, and it does not provide ways to discover or change its capacity independently of its size. Second, it provides a way for derived classes to discover when items have been added or removed. `List<T>` does not, on the grounds that it's your list so you presumably know when you add and remove

items. Notification mechanisms are not free, so `List<T>` avoids unnecessary overhead by not offering them. But `Collection<T>` assumes that external code will have access to your collection and that you will therefore not be in control of every addition and removal, justifying the overhead involved in providing a way for you to find out when the list is modified. (This is only available to the code deriving from `Collection<T>`. If you want code using your collection to be able to detect changes, the `ObservableCollection<T>` type is designed for that exact scenario. For example, if you use this type as the source for a list in desktop and mobile UI frameworks such as WPF or MAUI, they will be able to update the displayed list automatically when you modify the collection.)

You typically derive a class from `Collection<T>`, and you can override the virtual methods it defines to discover when the collection changes. ([Chapter 6](#) will discuss inheritance and overriding.) `Collection<T>` implements both `IList` and `IList<T>`, so you could present a `Collection<T>`-based collection through an interface type property, but it's common to make a derived collection type public and to use that instead of an interface as the property type.

ReadOnlyCollection<T>

If you want to provide a nonmodifiable collection, then instead of using `Collection<T>`, you can use `ReadOnlyCollection<T>`. This goes further than the restrictions imposed by arrays, by the way: not only can you not add, remove, or insert items, you cannot even replace elements. This class implements `IList<T>`, which requires an indexer with both a `get` and a `set`, but the `set` throws an exception. (Of course, it also implements `IReadOnlyCollection<T>`.)

If your collection's element type is a reference type, making the collection read-only does not prevent the objects to which the elements refer from being modified. I can retrieve, say, the 12th element from a read-only collection, and it will hand me back a reference. Fetching a reference counts as a read-only operation, but now that I have got that reference, the collection object is out of the picture, and I am free to do whatever I like with the reference. C# does not offer any concept of a read-only reference

for reference types,³ so the only way to present a truly read-only collection is to use an immutable type in conjunction with an `ICollection<T>` implementation.

There are two ways to use `ReadOnlyCollection<T>`. You can use it directly as a wrapper for an existing list—its constructor takes an `IList<T>`, and it will provide read-only access to that. (`List<T>` provides a method called `AsReadOnly` that constructs a read-only wrapper for you, by the way.) Alternatively, you could derive a class from it. As with `Collection<T>`, some classes do this for collections they wish to expose via properties, and it's usually because they want to define additional methods specific to the collection's purpose. Even if you derive from this class, you will still be using it to wrap an underlying list, because the only constructor it provides is the one that takes a list.

Addressing Elements with Index and Range Syntax

Whether using arrays, `List<T>`, `IList<T>`, or the various related types and interfaces just discussed, we've identified elements using simple examples such as `items[0]`, and more generally, expressions of the form `arrayOrListExpression[indexExpression]`. So far, all the examples have used an expression of type `int` for the index, but that is not the only choice. [Example 5-37](#) accesses the final element of an array using an alternative syntax.

Example 5-37. Accessing the last element of an array with an end-relative index

```
char[] letters = ['a', 'b', 'c', 'd'];  
char lastLetter = letters[^1];
```

This demonstrates one of two operators for use in indexers: the `^` operator. The other, shown in [Example 5-38](#), is the *range operator*. It consists of a pair of periods (`..`), and it is used to identify subranges of arrays, strings, or any indexable type that implements a certain pattern.

(Although this resembles the spread syntax we saw earlier, and also the slice syntax shown in [Chapter 2](#), those appear only in collection expressions and list patterns, respectively. This range syntax is something else.)

Example 5-38. Getting a subrange of an array with the range operator

```
int[] numbers = [1, 2, 3, 4, 5, 6, 7];  
// Gets 4th and 5th (but not the 3rd or 6th, for reasons explained shortly)  
int[] theFourthTheFifth = numbers[3..5];
```

Expressions using the `^` and `..` operators are of type `Index` and `Range`, respectively. These types are available in .NET but not .NET Framework, meaning that you can only use these language features on newer run-times.

System.Index

You can put the `^` operator in front of any `int` expression. It produces a `System.Index`, a value type that represents a position. When you create an index with `^`, it is end-relative, but you can also create start-relative indexes. There's no special operator for that, but since `Index` offers an implicit conversion from `int`, you can just assign `int` values directly into variables of type `Index`, as [Example 5-39](#) shows. You can also explicitly construct an index, as the line with `var` shows. The final `bool` argument is optional—it defaults to `false`—but I'm showing it to illustrate how `Index` knows which kind you want.

Example 5-39. Some start-relative and end-relative `Index` values

```
Index first = 0;  
Index second = 1;  
Index third = 2;  
var fourth = new Index(3, fromEnd: false);  
  
Index antePenultimate = ^3;  
Index penultimate = ^2;  
Index last = ^1;  
Index directlyAfterTheLast = ^0;
```

As [Example 5-39](#) shows, end-relative indexes exist independently of any particular collection. (Internally, `Index` stores end-relative indexes as negative numbers. This means that an `Index` is the same size as an `int`. It also means that negative start- or end-relative values are illegal—you'll get an exception if you try to create one.) C# generates code that determines the actual element position when you use an index. If `small` and `big` are arrays with 3 and 30 elements, respectively, `small[last]` would return the third, and `big[last]` would return the 30th. C# will turn these into `small[last.GetOffset(small.Length)]` and `big[last.GetOffset(big.Length)]`, respectively.

It has often been said that three of the hardest problems in computing are picking names for things and off-by-one errors. At first glance, [Example 5-39](#) makes it look like `Index` might be contributing to these problems. It may be vexing that the index for the third item is two, not three, but that at least is consistent with how arrays have always worked in C# and is normal for any zero-based indexing system. But given that zero-based convention, why on earth do the end-relative indexes appear to be one-based? We denote the first element with `0` but the last element with `^1`!

There are some good reasons for this. The fundamental insight is that in C#, indexes have always specified distances. When programming language designers choose a zero-based indexing system, this is not really a decision to call the first element 0: it is a decision to interpret an index as a distance from the start of an array. An upshot of this is that an index doesn't really refer to an item. [Figure 5-1](#) shows a collection with four elements and indicates where various index values would point in that collection. Notice that the indexes all refer to the boundaries between the items. This may seem like splitting hairs, but it's the key to understanding all zero-based index systems, and it is behind the apparent inconsistency in [Example 5-39](#).

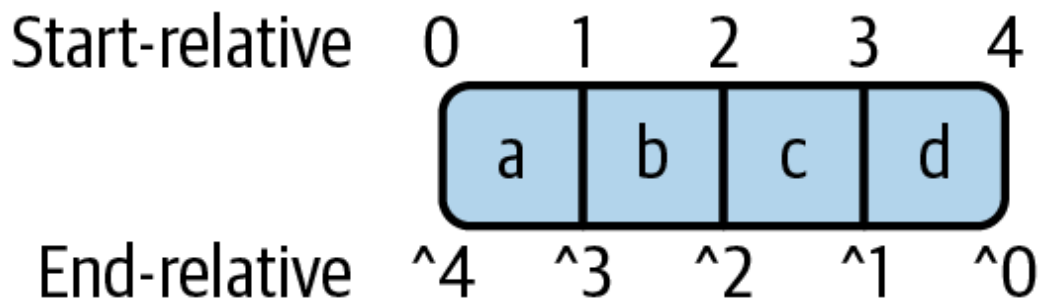


Figure 5-1. Where `Index` values point

When you access an element of a collection by index, you are asking for the element that *starts* at the position indicated by the index. So `array[0]` retrieves the single element that starts at the beginning of the array, the element that fills the space between indexes 0 and 1. Likewise, `array[1]` retrieves the element between indexes 1 and 2. What would `array[^0]` mean?⁴ That would be an attempt to fetch the element that *starts* at the very end of the array. Since elements all take up a certain amount of space, an element that starts at the very end of the array would necessarily finish one position after the end of the array. In this four-element array, `array[^0]` is equivalent to `array[4]`, so we're asking for the element occupying the space that starts four elements from the start and that ends five elements from the start. And since this is a four-element array, that's obviously not going to work.

The apparent discrepancy—the fact that `array[0]` gets the first, but we need to write `array[^1]` to get the last—occurs because elements sit between two indexes, and array indexers always retrieve the element between the index specified and the index after that. The fact that they do this even when you've specified an end-relative index is the reason those appear to be one-based. This language feature could have been designed differently: you could imagine a rule in which end-relative indexes always access the element that *ends* at the specified distance from the end and that starts one position earlier than that. There would have been a pleasing symmetry to this, because it would have made `array[^0]` refer to the final element, but this would have caused more problems than it solved.

It would be confusing to have indexers interpret an index in two different ways—it would mean that two different indexes might refer to the same position and yet fetch different elements. In any case, C# developers have

long been used to things working this way. As [Example 5-40](#) shows, the way to access the final element of an array before the `^` index operator was added was to use an index calculated by subtracting one from the length. And if you wanted the element before last, you subtracted two from the length, and so on. As you can see, the new end-relative syntax is entirely consistent with the older approach.

Example 5-40. End-relative indexing without and with `Index`

```
int lastOld = numbers[numbers.Length - 1];
int lastNew = numbers[^1];

int penultimateOld = numbers[numbers.Length - 2];
int penultimateNew = numbers[^2];
```

One more way to think of this is to wonder what it might look like if we accessed arrays by specifying ranges. The first element is in the range 0–1, and the last is in the range `^1–^0`. Expressed this way, there is clearly symmetry between the start-relative and end-relative forms. And speaking of ranges...

System.Range

As I said earlier, C# has two operators useful for working with arrays and other indexable types. We've just looked at `^` and the corresponding `Index` type. The other is called the *range operator*, and it has a corresponding type, `Range`, also in the `System` namespace. A `Range` is a pair of `Index` values, which it makes available through `Start` and `End` properties. `Range` offers a constructor taking two `Index` values, but in C# the idiomatic way to create one is with the range operator, as [Example 5-41](#) shows.

Example 5-41. Various ranges

```
Range everything = 0..^0;
Range alsoEverything = 0..;
Range everythingAgain = ..^0;
Range everythingOneMoreTime = ..;
```

```
var yetAnotherWayToSayEverything = Range.All;

Range firstThreeItems = 0..3;
Range alsoFirstThreeItems = ..3;

Range allButTheFirstThree = 3..0;
Range alsoAllButTheFirstThree = 3..;

Range allButTheLastThree = 0..3;
Range alsoAllButTheLastThree = ..3;

Range lastThreeItems = 3..0;
Range alsoLastThreeItems = 3..;
```

As you can see, if you do not put a start index before the `..`, it defaults to 0, and if you omit the end, it defaults to `^0` (i.e., the very start and end, respectively). The example also shows that the start can be either start-relative or end-relative, as can the end.

WARNING

The default value for `Range`—the one you’ll get in a field or array element that you do not explicitly initialize—is `0..0`. This denotes an empty range. While this is a natural upshot of the fact that value types are always initialized to zero-like values by default, it might not be what you’d expect given that `..` is equivalent to `Range.All`.

Since `Range` works in terms of `Index`, the start and end denote offsets, not elements. For example, consider what the range `1..3` would mean for the elements shown in [Figure 5-1](#). In this case, both indexes are start-relative. The start index, `1`, is the boundary between the first and second elements (`a` and `b`), and the end index, `3`, is the boundary between the third and fourth elements (`c` and `d`). So this is a range that starts at the beginning of `b` and ends at the end of `c`, as [Figure 5-2](#) shows. So this identifies a two-element range (`b` and `c`).

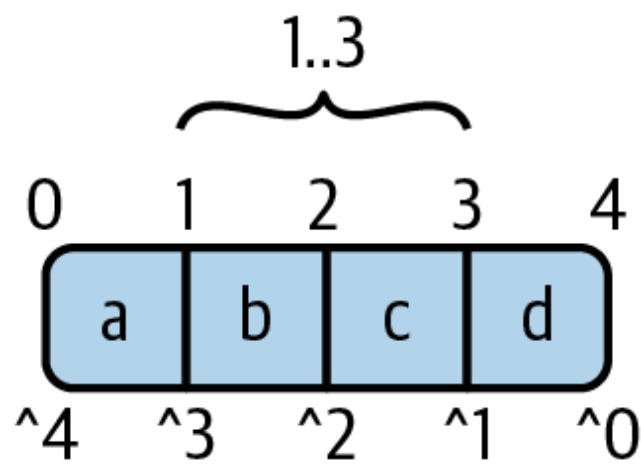


Figure 5-2. A range

The interpretation of ranges sometimes surprises people when they first see it: some expect `1..3` to represent the first, second, and third elements (or, if they take into account C#'s zero-based indexing, perhaps the second, third, and fourth elements). It can seem inconsistent at first that the start index appears to be inclusive while the end index is exclusive. But once you remember that an index refers not to an item but to an offset, and therefore the boundary between two items, it all makes sense. If you draw the positions represented by a range's indexes, as [Figure 5-2](#) does, it becomes perfectly obvious that the range identified by `1..3` covers just two elements.

So what can we do with a `Range`? As [Example 5-38](#) showed, we can use one to get a subrange of an array. That creates a new array of the relevant size and copies the values from the range into it. This same syntax also works for getting substrings, as [Example 5-42](#) shows.

Example 5-42. Getting a substring with a range

```
string t1 = "dysfunctional";  
string t2 = t1[3..6];  
Console.WriteLine($"Putting the {t2} in {t1}");
```

You can also use `Range` with `ArraySegment<T>`, a value type that refers to a range of elements within an array. [Example 5-43](#) makes a slight modification to [Example 5-38](#). Instead of passing the range to the array's indexer, this first creates an `ArraySegment<int>` that represents the entire array, and then uses a range to get a second `ArraySegment<int>` repre-

senting the fourth and fifth elements. The advantage of this is that it does not need to allocate a new array—both `ArraySegment<int>` values refer to the same underlying array; they just point to different sections of it, and since `ArraySegment<int>` is a value type, this can avoid allocating new heap blocks. (`ArraySegment<int>` has no direct support for range, by the way. The compiler turns this into a call to the segment's `Slice` method.)

Example 5-43. Getting a subrange of an `ArraySegment<T>` with the range operator

```
int[] numbers = [1, 2, 3, 4, 5, 6, 7];
ArraySegment<int> wholeArrayAsSegment = numbers;
ArraySegment<int> theFourthTheFifth = wholeArrayAsSegment[3..5];
```

The `ArraySegment<T>` type has been around since .NET 2.0 (and has been in .NET Standard since 1.0). It's a useful way to avoid extra allocations, but it's limited: it only works with arrays. What about strings? All current versions of .NET support types offering a more general form of this concept, `Span<T>` and `ReadOnlySpan<T>`. (On .NET Framework, these are available through the `System.Memory` NuGet package. They are built into other .NET versions.) Just like `ArraySegment<T>`, `Span<T>` represents a subsequence of items inside something else, but it is much more flexible about what that “something else” might be. It could be an array, but it can also be a string, memory in a stack frame, or memory allocated by some library or system call entirely outside of .NET. The `Span<T>` and `ReadOnlySpan<T>` types are discussed in more detail in [Chapter 18](#), but for now, [Example 5-44](#) illustrates their basic use.

Example 5-44. Getting a subrange of a span with the range operator

```
int[] numbers = [1, 2, 3, 4, 5, 6, 7];
Span<int> wholeArrayAsSpan = numbers;
Span<int> theFourthTheFifth = wholeArrayAsSpan[3..5];
ReadOnlySpan<char> textSpan = "dysfunctional";
ReadOnlySpan<char> such = textSpan[3..6];
```

These last two examples have much the same logical meaning as the preceding examples, but they avoid making copies of the underlying data.

We've now seen that we can use ranges with several types: arrays, strings, `ArraySegment<T>`, `Span<T>`, and `ReadOnlySpan<T>`. This raises a question: Does C# have a list of types that get special handling, or can we support indexers and ranges in our own types? The answers are, respectively, yes and yes. C# has some baked-in handling for arrays and strings: it knows to call specific runtime library methods to produce subarrays and substrings. However, there is no special range handling for array segments or spans: they work because they conform to a pattern. There is also a pattern to enable use of `Index`. If you support these same patterns, you can make `Index` and `Range` work with your own types.

Supporting Index and Range in Your Own Types

The array type does not define an indexer that accepts an argument of type `Index`. Nor do any of the generic array-like types shown earlier in this chapter—they all just have ordinary `int`-based indexers; however, you can use `Index` with them nonetheless. And as I explained earlier, code of the form `a[index]` will expand to `a[index.GetOffset(a.Length)]`.⁵ So all you need is an `int`-based indexer and a property of type `int` called either `Length` or `Count`.

[Example 5-45](#) shows about the least amount of work you can possibly do to enable code to pass an `Index` to your type's indexer. It's not a very useful implementation, but it's enough to keep C# happy.

Example 5-45. Minimally enabling `Index`

```
public class Indexable
{
    public char this[int index] => (char)('0' + index);

    public int Length => 10;
}
```

There's an even simpler way: just define an indexer that takes an argument of type `Index`. However, most indexable types supply an `int`-

based indexer, so in practice you'd be overloading your indexer, offering both forms. That is not simpler, but it would enable your code to distinguish between start- and end-relative indexes. If we use either `1` or `^9` with [Example 5-45](#), its indexer sees `1` in either case, because C# generates code that converts the `Index` to a start-based `int`, but if you write an indexer with an `Index` parameter, C# will pass the `Index` straight in. If you overload the indexer so that both `int` and `Index` forms are available, it will never generate code that converts an `Index` to an `int` in order to call the `int` indexer: the pattern only kicks in if no `Index`-specific indexer is available.

`ICollection<T>` meets the pattern's requirements (as do types that implement it, such as `List<T>`), so you can pass an `Index` to the indexer of anything that implements this. (It supplies a `Count` property instead of `Length`, but the pattern accepts either.) This is a widely implemented interface, so in practice, many types automatically get support for `Index` despite having been written before `Index` was introduced. This is an example of how the pattern-based support for `Index` means libraries that target older .NET versions (such as .NET Standard 2.0) where `Index` is not available can nonetheless define types that will work with `Index` when used with newer versions of .NET.

The pattern for supporting `Range` is different: if your type supplies an instance method called `Slice` that takes two integer arguments, C# will allow code to supply a `Range` as an indexer argument. [Example 5-46](#) shows the least a type can do to enable this, although it's not a very useful implementation. (As with `Index`, you can alternatively just define an indexer overload that accepts a `Range` directly. But again, an advantage to the pattern approach is that you can use it when targeting older versions that do not offer the `Range` or `Index` types—such as .NET Standard 2.0—while still supporting ranges for code that targets newer versions.)

Example 5-46. Minimally enabling `Range`

```
public class Rangeable
{
    public int Length => 10;
```



```
public Rangeable Slice(int offset, int length) => this;
}
```

You might have noticed that this type doesn't define an indexer. That's because this pattern-based support for expressions of the form `x[1..^1]` doesn't need one. It may look like that expression uses an indexer, but this just calls the `Slice` method. (Likewise, the earlier range examples with `string` and arrays compile into method calls.) You need the `Length` property (or `Count`) because the compiler generates code that relies on this to resolve the range's indexes. [Example 5-47](#) shows roughly how the compiler uses types that support this pattern.

Example 5-47. How range indexing expands

```
Rangeable r1 = new();
Range r = 2..^2;

Rangeable r2;

r2 = r1[r];
// is equivalent to
int startIndex = r.Start.GetOffset(r1.Length);
int endIndex = r.End.GetOffset(r1.Length);
r2 = r1.Slice(startIndex, endIndex - startIndex);
```

So far, all of the collections we've looked at have been linear: I've shown only simple sequences of objects or values, some of which offer indexed access. However, .NET provides other kinds of collections.

Dictionaries

One of the most useful kinds of collections is a dictionary. .NET offers the `Dictionary<TKey, TValue>` class, and there's a corresponding interface called, predictably, `IDictionary<TKey, TValue>`, and also a read-only version, `IReadOnlyDictionary<TKey, TValue>`. These represent collections of key/value pairs, and their most important capability is to look up a value based on its key, making dictionaries useful for representing associations.

Suppose you are writing a UI for an application that supports online discussions. When displaying a message, you might want to show certain things about the user who sent it, such as their name and picture, and you'd probably want to avoid fetching these details from a persistent store every time; if the user is in conversation with a few friends, the same people will crop up repeatedly, so you'd want some sort of cache to avoid duplicate lookups. You might use a dictionary as part of this cache. [Example 5-48](#) shows an outline of this approach. (It omits application-specific details of how the data is actually fetched and when old data is removed from memory.)

Example 5-48. Using a dictionary as part of a cache

```
public class UserCache
{
    private readonly Dictionary<string, UserInfo> _cachedUserInfo = new();

    public UserInfo GetInfo(string userHandle)
    {
        RemoveStaleCacheEntries();
        if (!_cachedUserInfo.TryGetValue(userHandle, out UserInfo? info))
        {
            info = FetchUserInfo(userHandle);
            _cachedUserInfo.Add(userHandle, info);
        }
        return info;
    }

    private UserInfo FetchUserInfo(string userHandle)
    {
        // fetch info...
    }

    private void RemoveStaleCacheEntries()
    {
        // application-specific logic deciding when to remove old entries...
    }
}

public class UserInfo
{

```

```
// application-specific user information...  
}
```

The first type argument, `TKey`, is used for lookups, and in this example, I'm using a string that identifies the user in some way. The `TValue` argument is the type of value associated with the key—information previously fetched for the user and cached locally in a `UserInfo` instance, in this case. The `GetInfo` method uses `TryGetValue` to look in the dictionary for the data associated with a user handle. There is a simpler way to retrieve a value. As [Example 5-49](#) shows, dictionaries provide an indexer. However, that throws a `KeyNotFoundException` if there is no entry with the specified key. That would be fine if your code always expects to find what it's looking for, but in our case, the key will be missing for any user whose data is not already in the cache. This will probably happen rather a lot, which is why I'm using `TryGetValue`. As an alternative, we could have used the `ContainsKey` method to see if the entry exists before retrieving it, but that's inefficient if the value is present—the dictionary would end up looking up the entry twice, once in the call to `ContainsKey` and then again when we use the indexer. `TryGetValue` performs the test and the lookup as a single operation.

Example 5-49. Dictionary lookup with indexer

```
UserInfo info = _cachedUserInfo[userHandle];
```

As you might expect, we can also use the indexer to set the value associated with a key. I've not done that in [Example 5-48](#). Instead, I've used the `Add` method, because it has subtly different semantics: by calling `Add`, you are indicating that you do not think any entry with the specified key already exists. Whereas the dictionary's indexer will silently overwrite an existing entry if there is one, `Add` will throw an exception if you attempt to use a key for which an entry already exists. In situations where the presence of an existing key would imply that something is wrong, it's better to call `Add` so that the problem doesn't go undetected.

The `IDictionary<TKey, TValue>` interface requires its implementations also to provide the `ICollection<KeyValuePair<TKey, TValue>>` interface, and therefore also `IEnumerable<KeyValuePair<TKey, TValue>>`.

The read-only counterpart requires the latter but not the former. These interfaces depend on a generic struct, `KeyValuePair<TKey, TValue>`, which is a simple container that wraps a key and a value in a single instance. This means you can iterate through a dictionary using `foreach`, and it will return each key/value pair in turn.

The presence of an `IEnumerable<T>` and an `Add` method also means that we can use the collection initializer syntax. It's not quite the same as with a simple list, because a dictionary's `Add` takes two arguments: the key and value. However, the collection initializer syntax can cope with multi-argument `Add` methods. You wrap each set of arguments in nested braces, as [Example 5-50](#) shows.

Example 5-50. Collection initializer syntax with a dictionary

```
var textToNumber = new Dictionary<string, int>
{
    { "One", 1 },
    { "Two", 2 },
    { "Three", 3 },
};
```

As you saw in [Chapter 3](#), there's an alternative way to populate a dictionary: instead of using a collection initializer, you can use the object initializer syntax. As you may recall, this syntax lets you set properties on a newly created object. Indexers are just a special kind of property, so it makes sense to be able to set them with an object initializer. Although [Chapter 3](#) showed this already, it's worth comparing object initializers with collection initializers, so [Example 5-51](#) shows the alternative way to initialize a dictionary.

Example 5-51. Object initializer syntax with a dictionary

```
var textToNumber = new Dictionary<string, int>
{
    ["One"] = 1,
    ["Two"] = 2,
```

```
["Three"] = 3  
};
```

Although the outcome is the same here with Examples [5-50](#) and [5-51](#), the compiler generates slightly different code for each. With [Example 5-50](#), it populates the collection by calling `Add`, whereas [Example 5-51](#) uses the indexer. In these examples the result is the same, so there's no objective reason to choose one over the other, but the difference could matter in some cases. For example, if you are using a class that has an indexer but no `Add` method, only the index-based code would work. And even with `Dictionary<TKey, TValue>`, which offers both, `Add` throws an exception if you supply duplicate keys, whereas the indexer does not, so the collection initializer syntax has the benefit of checking that your keys are unique.

NOTE

You can use the collection expression syntax added in C# 12.0 to initialize an empty dictionary, but it does not provide a way to add entries. Currently, only the older initializer syntax can do this.

The `Dictionary<TKey, TValue>` collection class relies on hashes to offer fast lookup. [Chapter 3](#) described the `GetHashCode` method, and you should ensure that whatever type you are using as a key provides a good hash implementation. The `string` class overrides `GetHashCode` so that it works well as a key, for example. The default `GetHashCode` method is viable only if different instances of a type are always considered to have different values, but types for which that is true function well as keys. Alternatively, the dictionary class provides constructors that accept an `IEqualityComparer<TKey>`, which allows you to provide an implementation of `GetHashCode` and `Equals` to use instead of the one supplied by the key type itself. [Example 5-52](#) uses this to make a case-insensitive version of [Example 5-50](#).

Example 5-52. A case-insensitive dictionary

```
var textToNumber =  
    new Dictionary<string, int>(StringComparer.InvariantCultureIgnoreCase)  
{  
    { "One", 1 },  
    { "Two", 2 },  
    { "Three", 3 },  
};
```

This uses the `StringComparer` class, which provides various implementations of `IComparer<string>` and `IEqualityComparer<string>`, offering different comparison rules. Here, I've chosen an ordering that ignores case, so `textToNumber["tHrEe"]` would have the same effect as `textToNumber["three"]`. This particular comparer also ignores the configured locale, ensuring consistent behavior in different regions. If I were using strings to be displayed, I'd probably use one of its culture-aware comparisons.

Sorted Dictionaries

`Dictionary<TKey, TValue>` does not guarantee to provide elements in any particular order when you iterate over its contents. In simple cases, it might return items in the order in which they were added, but you should not depend on this.

Sometimes, it's useful to be able to retrieve the contents of a dictionary in some meaningful order. You could always get the contents into an array and then sort them, but the `System.Collections.Generic` namespace contains two more implementations of the `IDictionary<TKey, TValue>` interface that keep their contents permanently in order. There's `SortedDictionary<TKey, TValue>` and the more confusingly titled `SortedList<TKey, TValue>`, which—despite the name—implements the `IDictionary<TKey, TValue>` interface and does not directly implement `IList<T>`.

These classes do not use hash codes. They still provide reasonably fast lookup, but they do it by keeping their contents sorted. They maintain the order every time you add a new entry, which makes addition rather slower for both these classes than with the hash-based dictionary, but it means that when you iterate over the contents, they come out in order. As with array and list sorting, you can specify custom comparison logic, but if you don't supply that, these dictionaries require the key type to implement `Comparable<T>`.

The ordering maintained by a `SortedDictionary<TKey, TValue>` is apparent only when you use its enumeration support (e.g., with `foreach`). `SortedList<TKey, TValue>` also enumerates its contents in order, but it additionally provides numerically indexed access to the keys and values. This does not work through the object's indexer—that expects to be passed a key just like any dictionary. Instead, the sorted list dictionary defines two properties, `Keys` and `Values`, which provide all the keys and values as `IList<TKey>` and `IList<TValue>`, respectively, sorted so that the keys will be in ascending order. (The `Values` are in key order as well as the `Keys`.)

Inserting and removing objects is relatively expensive for the sorted list because it has to shuffle the key and value list contents up or down. (This means a single insertion has $O(n)$ complexity.) The sorted dictionary, on the other hand, uses a tree data structure to keep its contents sorted. The exact details are not specified, but insertion and removal performance are documented as having $O(\log n)$ complexity, which is much better than for the sorted list.⁶ However, this more complex data structure gives a sorted dictionary a significantly larger memory footprint. This means that neither is definitively faster or better than the other—it all depends on the usage pattern, which is why the runtime libraries supply both.

In most cases, the hash-based `Dictionary<TKey, Value>` will provide better insertion, removal, and lookup performance than either of the sorted dictionaries, and much lower memory consumption than a `SortedDictionary<TKey, TValue>`, so you should use these sorted dictionary collections only if you need to access the dictionary's contents in order.

Sets

The `System.Collections.Generic` namespace defines an `ISet<T>` interface. This offers a simple model: a particular value is either a member of the set or not. You can add or remove items, but a set does not keep track of how many times you've added an item, nor does `ISet<T>` require items to be stored in any particular order.

All set types implement `ICollection<T>`, which provides the methods for adding and removing items. In fact, it also defines the method for determining membership: although I've not drawn attention to it before now, you can see in [Example 5-29](#) that `ICollection<T>` defines a `Contains` method. This takes a single value and returns `true` if that value is in the collection.

Given that `ICollection<T>` already provides the defining operations for a set, you might wonder why we need `ISet<T>`. But it does add a few things. Although `ICollection<T>` defines an `Add` method, `ISet<T>` defines its own subtly different version, which returns a `bool`, so you can find out whether the item you just added was already in the set.

[Example 5-53](#) uses this to detect duplicates in a method that displays each string in its input once. (This illustrates the usage, but in practice it would be simpler to use the `Distinct` LINQ operator described in [Chapter 10](#).)

Example 5-53. Using a set to determine what's new

```
public static void ShowEachDistinctString(IEnumerable<string> strings)
{
    var shown = new HashSet<string>(); // Implements ISet<T>
    foreach (string s in strings)
    {
        if (shown.Add(s))
        {
            Console.WriteLine(s);
        }
    }
}
```


`ISet<T>` also defines some operations for combining sets. The `UnionWith` method takes an `IEnumerable<T>` and adds to the set all the values from that sequence that were not already in the set. The `ExceptWith` method removes from the set items that are also in the sequence you pass. The `IntersectWith` method removes from the set items that are not also in the sequence you pass. And `SymmetricExceptWith` also takes a sequence and removes from the set elements that are in the sequence, but also adds to the set values in the sequence that were not previously in the set.

There are also some methods for comparing sets. Again, these all take an `IEnumerable<T>` argument representing the other set with which the comparison is to be performed. `IsSubsetOf` and `IsProperSubsetOf` both let you check whether the set on which you invoke the method contains only elements that are also present in the sequence, with the latter method additionally requiring the sequence to contain at least one item not present in the set. `IsSupersetOf` and `IsProperSupersetOf` perform the same tests in the opposite direction. The `Overlaps` method tells you whether the two sets share at least one element in common.

Mathematical sets do not define an order for their contents, so it's not meaningful to refer to the 1st, 10th, or *n*th element of a set—you can ask only whether an element is in the set or not. In keeping with this feature of mathematical sets, .NET sets do not support indexed access, so `ISet<T>` does not demand support for `IList<T>`. Sets are free to produce the members in whatever order they like in their `IEnumerable<T>` implementation.

The runtime libraries offer classes that provide this interface with different implementation strategies, including: `HashSet` and `SortedSet`. As you may have guessed from the names, one of these does in fact choose to keep its elements in order; `SortedSet` keeps its contents sorted at all times and presents items in this order through its `IEnumerable<T>` implementation. The documentation does not describe the exact strategy used to maintain the order, but it appears to use a balanced binary tree to support efficient insertion and removal, and to offer fast lookup when trying to determine whether a particular value is already in the list.

`HashSet` works more like `Dictionary<TKey, TValue>`. It uses hash-based lookup, which can often be faster than the ordered approach, but if you enumerate through the collection with `foreach`, the results will not be in any useful order. (So the relationship between `HashSet` and `SortedSet` is much like that between the hash-based dictionary and the sorted dictionaries.)

Queues and Stacks

A *queue* is a list where you can only add items to the end of the list, and you can only remove the first item (at which point the second item, if there was one, becomes the new first item). This style of list is often called a first-in, first-out (FIFO) list. This makes it less useful than a `List<T>`, because you can read, write, insert, or remove items at any point in a `List<T>`. However, the constraints make it possible to implement a queue with considerably better performance characteristics for insertion and removal. When you remove an item from a `List<T>`, it has to shuffle all the items after the one removed to close up the gap, and insertions require a similar shuffle. Insertion and removal at the end of a `List<T>` is efficient, but if you need FIFO semantics, you can't work entirely at the end—you'll need to do either insertions or removals at the start, making `List<T>` a bad choice. `Queue<T>` can use a much more efficient strategy because it needs only to support queue semantics. (It uses a circular buffer internally, although that's an undocumented implementation detail.)

To add a new item to the end of a queue, call the `Enqueue` method. To remove the item at the head of the queue, call `Dequeue`, or use `Peek` if you want to look at the item without removing it. Both operations will throw an `InvalidOperationException` if the queue is empty. You can find out how many items are in the queue with the `Count` property.

Although you cannot insert, remove, or change items in the middle of the list, you can inspect the whole queue, because `Queue<T>` implements `IEnumerable<T>` and also provides a `ToArray` method that returns an array containing a copy of the current queue contents. It also implements `ICollection<T>`, so you could instead call the `CopyTo` method, which copies the queue contents into an array that you supply.

A *stack* is similar to a queue, except you retrieve items from the same end as you insert them—so this is a last-in, first-out (LIFO) list. `Stack<T>` looks very similar to `Queue<T>` except instead of `Enqueue` and `Dequeue`, the methods for adding and removing items use the traditional names for stack operations: `Push` and `Pop`. (Other methods, such as `Peek` and `ToArray`, remain the same.)

The runtime libraries do not offer a double-ended queue. However, linked lists can offer a superset of that functionality.

Linked Lists

The `LinkedList<T>` class provides an implementation of the classic doubly linked list data structure, in which each item in the sequence is wrapped in an object (of type `LinkedListNode<T>`) that provides a reference to its predecessor and its successor. The advantage of a linked list is that insertion and removal is inexpensive—it does not require elements to be moved around in arrays and does not require binary trees to be re-balanced. It just requires a few references to be swapped around. The downsides are that linked lists have fairly high memory overheads, requiring an extra object on the heap for every item in the collection, and it's also relatively expensive for the CPU to get to the n th item because you have to go to the start and then traverse n nodes.

The first and last nodes in a `LinkedList<T>` are available through the predictably named `First` and `Last` properties. You can insert items at the start or end of the list with `AddFirst` and `AddLast`, respectively. To add items in the middle of a list, call either `AddBefore` or `AddAfter`, passing in the `LinkedListNode<T>` before or after which you'd like to add the new item.

The list also provides `RemoveFirst` and `RemoveLast` methods and two overloads of a `Remove` method that allow you to remove either the first node that has a specified value or a particular `LinkedListNode<T>`.

The `LinkedListNode<T>` itself provides a `Value` property of type `T` containing the actual item for this node's point in the sequence. Its `List`

property refers back to the containing `LinkedList<T>`, and the `Previous` and `Next` properties allow you to find the previous or next node.

To iterate through the contents of a linked list, you could retrieve the first node from the `First` property and then follow each node's `Next` property until you get a `null`. However, `LinkedList<T>` implements `IEnumerable<T>`, so it's easier just to use a `foreach` loop. If you want to get the elements in reverse order, start with `Last` and follow each node's `Previous`. If the list is empty, `First` and `Last` will be `null`.

Concurrent Collections

The collection classes described so far are designed for single-threaded usage. You are free to use different instances on different threads simultaneously, but a particular instance of any of these types must be used only from one thread at any one time.⁷ But some types are designed to be used by many threads simultaneously, without needing to use the synchronization mechanisms discussed in [Chapter 16](#). These are in the `System.Collections.Concurrent` namespace.

The concurrent collections do not offer equivalents for every nonconcurrent collection type. Some classes are designed to solve specific concurrent programming problems. Even with the ones that do have nonconcurrent counterparts, the need for concurrent use without locking can mean that they present a somewhat different API than any of the normal collection classes.

The `ConcurrentQueue<T>` and `ConcurrentStack<T>` classes are the ones that look most like the nonconcurrent collections we've already seen, although they are not identical. The queue's `Dequeue` and `Peek` have been replaced with `TryDequeue` and `TryPeek`, because in a concurrent world, there's no reliable way to know in advance whether attempting to get an item from the queue will succeed. (You could check the queue's `Count`, but even if that is nonzero, some other thread may get in there and empty the queue between when you check the count and when you attempt to retrieve an item.) So the operation to get an item has to be atomic with

the check for whether an item is available, hence the `Try` forms that can fail without throwing an exception. Likewise, the concurrent stack provides `TryPop` and `TryPeek`.

`ConcurrentDictionary<TKey, TValue>` looks fairly similar to its nonconcurrent cousin, but it adds some extra methods to provide the atomicity required in a concurrent world. For example, the `GetOrAdd` method combines the test for the presence of a key with the addition of a new entry as a single atomic operation. It returns the existing value if there is one, and it can invoke a function to get the new value when required.

There is no concurrent list, because you tend to need more coarse-grained synchronization to use ordered, indexed lists successfully in a concurrent world. But if you just want a bunch of objects, there's `ConcurrentBag<T>`, which does not maintain any particular order. (Although the lack of ordering makes this sound similar to a set, there's one important difference: a bag can contain multiple copies of the same item.)

There's also `BlockingCollection<T>`, which acts like a queue but allows threads that want to take items off the queue to choose to block until an item is available. You can also set a limited capacity and make threads that put items onto the queue block if the queue is currently full, waiting until space becomes available.

Immutable Collections

Microsoft provides a set of collection classes that guarantee immutability and yet provide a lightweight way to produce a modified version of the collection without having to make an entire new copy. (These are built into .NET, but in .NET Framework, you will need a reference to the `System.Collections.Immutable` NuGet package to use these.)

Immutability can be a very useful characteristic in multithreaded environments, because if you know that the data you are working with cannot change, you don't need to take special precautions to synchronize your access to it. (This is a stronger guarantee than you get with

`ReadOnlyList<T>`, which merely prevents you from modifying the collection; it could just be a façade over a collection that some other thread is able to modify.) But what do you do if your data needs to be updated occasionally? It seems a shame to give up on immutability and to take on the overhead of traditional multithreaded synchronization in cases where you expect conflicts to be rare.

A low-tech approach is to build a new copy of all of your data each time something changes (e.g., when you want to add an item to a collection, create a whole new collection with a copy of all the old elements and also the new one, and use that new collection from then on). The built-in `string` type works in exactly the same way because it is also immutable—the methods that sound like they will change the value, such as `Trim`, actually return a new string. This works but can be extremely inefficient when working with large quantities of data. However, techniques exist that can effectively reuse parts of existing collections. The basic principle is that if you want to add an item to a collection, you build a new collection that just points to the data that is already there, along with some extra information to say what has changed. It is rather more complex in practice, but the key point is that there are well-established ways in which to implement various kinds of collections so that you can efficiently build what look like complete self-contained copies of the original data with some small modification applied, without either having to modify the original data or having to build a complete new copy of the collection. The immutable collections do all this for you, encapsulating the work behind some straightforward interfaces.

This enables a model where you're free to update your application's model without affecting code that was in the middle of using the current version of the data. Consequently, you don't need to hold locks while reading data—you might need some synchronization when getting the latest version of the data, but thereafter, you can process the data without any concurrency concerns. This can be especially useful when writing multithreaded code. The .NET Compiler Platform (often known by its codename, Roslyn) that is the basis of Microsoft's C# compiler uses this technique to enable compilation to exploit multiple CPU cores efficiently.

The `System.Collections.Immutable` namespace defines its own interfaces—`IImmutableList<T>`, `IImmutableDictionary<TKey, TValue>`, `IImmutableQueue<T>`, `IImmutableStack<T>`, and `IImmutableSet<T>`. These collections can't just implement the interfaces we saw earlier, because all operations that modify the collection in any way need to return a new collection. [Example 5-54](#) shows what this means for adding entries to a dictionary.

Example 5-54. Creating immutable dictionaries

```
IImmutableDictionary<int, string> d = ImmutableDictionary.Create<int, string>();  
d = d.Add(1, "One");  
d = d.Add(2, "Two");  
d = d.Add(3, "Three");
```

The whole point of immutable types is that code using an existing object can be certain that nothing will change, so additions, removals, or modifications necessarily mean creating a new object that looks just like the old one but with the modification applied. So in [Example 5-54](#), the variable `d` refers successively to four different immutable dictionaries: an empty one, one with one value, one with two values, and finally one with all three values.

If you are adding a range of values like this, and you won't be making intermediate results available to other code, it is more efficient to add multiple values in a single operation, because it doesn't have to produce a separate `IImmutableDictionary<TKey, TValue>` for each entry you add. (You could think of immutable collections as working a bit like a source control system, with each change corresponding to a commit—for every commit you do, a version of the collection will exist that represents its contents immediately after that change.) It's more efficient to batch a bunch of related changes into a single “version” so the collections all have `AddRange` methods that let you add multiple items in one step.

When you're building a new collection from scratch, the same principle applies: it will be more efficient if you put all of the initial content into the first version of the collection, instead of adding items one at a time. Each immutable collection type offers a nested `Builder` class to make

this easier, enabling you to add items one at a time but to defer the creation of the actual collection until you have finished. [Example 5-55](#) shows how this is done.

Example 5-55. Creating an immutable dictionary with a builder

```
ImmutableDictionary<int, string>.Builder b =  
    ImmutableDictionary.CreateBuilder<int, string>();  
b.Add(1, "One");  
b.Add(2, "Two");  
b.Add(3, "Three");  
ImmutableDictionary<int, string> d = b.ToImmutable();
```

The builder object is not immutable. Much like `StringBuilder`, it is a mutable object that provides an efficient way to build a description of an immutable object.

All of the immutable collection types except for dictionaries support the collection expression syntax introduced in C# 12.0. [Example 5-56](#) uses this to create an immutable list.

Example 5-56. Creating an immutable list with a collection expression

```
ImmutableList<int> numbers = [1, 2, 3, 4, 5];
```

In addition to the immutable list, dictionary, queue, stack, and set types, there's one more immutable collection class that is a bit different than the rest: `ImmutableArray<T>`. This is essentially a wrapper providing an immutable façade around an array. It implements `IImmutableList<T>`, meaning that it offers the same services as an immutable list, but it has quite different performance characteristics.

When you call `Add` on an immutable list, it will attempt to reuse most of the data that is already there, so if you have a million items in your list, the “new” list returned by `Add` won't contain a new copy of those items—it will mostly reuse the data that was already there. However, to achieve this, `ImmutableList<T>` uses a somewhat complex tree data structure in-

ternally. The upshot is that looking up values by index in an `ImmutableList<T>` is not nearly as efficient as using an array (or a `List<T>`). The indexer for `ImmutableList<T>` has $O(\log n)$ complexity.

An `ImmutableArray<T>` is much more efficient for reads—being a wrapper around an array, it has $O(1)$ complexity, i.e., the time taken to fetch an entry is constant, regardless of how large the collection may be. The trade-off is that all of the `IImmutableList<T>` methods for building a modified version of the list (`Add`, `Remove`, `Insert`, `SetItem`, etc.) build a complete new array, including a new copy of any data that needs to be carried over. (In other words, unlike all the other immutable collection types, `ImmutableArray<T>` employs the low-tech approach to immutability that I described earlier.) This makes modifications very much more expensive, but if you have some data you do not expect to modify after the initial creation of the array, this is an excellent trade-off, because you will only ever build one copy of the array. And if you need to make only very occasional modifications, the high cost of each change might still be worth it overall.

Frozen Collections

.NET 8.0 adds a new namespace, `System.Collection.Frozen`, which defines `FrozenDictionary<TKey, TValue>` and `FrozenSet<T>` types. These are immutable implementations of the dictionary and set interfaces, respectively, and they are optimized for a different usage model than the corresponding immutable collections shown in the preceding section. Whereas those offer methods that produce a modified version of the collection without having to make an entire new copy, the frozen collections are optimized for scenarios in which you decide up front what you want in your collection, and will not make any further changes.

Despite the name, the immutable collection types are designed to accommodate change; they just do this by providing snapshots, and there are overheads involved in enabling change while making each individual snapshot immutable. That makes these types suboptimal in cases where there will be no further changes. (`ImmutableArray<T>` is an exception.) The ordinary, modifiable collection types such as `Dictionary<TKey,`

TValue>` also need to be able to accommodate change, which also comes at a cost—they typically preallocate extra memory in anticipation of more items being added, for example. The frozen collection types are free to optimize their internal storage for exactly one data set because they don't need to be able to support subsequent change. They can do more work during initialization to improve the performance of each lookup.

The runtime libraries define extension methods for creating instances of the frozen collection types from other collections. [Example 5-57](#) shows how to obtain a frozen dictionary from a normal one. There's a similar `ToFrozenSet` extension method that can convert any `IEnumerable<T>` into a `FrozenSet<T>`.

Example 5-57. Creating a `FrozenDictionary<TKey, TValue>` from a `Dictionary<TKey, TValue>`

```
Dictionary<int, string> ordinary = GetOrdinaryDictionary();  
FrozenDictionary<int, string> frozen = ordinary.ToFrozenDictionary();
```

`FrozenDictionary` implements `IDictionary<TKey, TValue>` and `ReadOnlyDictionary<TKey, TValue>`, so once you've created one you can use it like a normal dictionary. (It implements the mutable `IDictionary<TKey, TValue>` because some libraries require that interface even when they won't actually modify the dictionary, but its implementation will throw a `NotSupportedException` if you try to modify the collection.)

The frozen collection types won't always be faster. In scenarios where they are a good fit—when you can supply the entire contents up front—they will usually outperform the immutable collections, but they won't necessarily outperform the ordinary collection types such as `Dictionary<TKey, TValue>`. The frozen collections do extra work during initialization in exchange for making lookups faster, so they only offer an improvement if you perform enough lookups to offset that initial cost. As with any performance-oriented change you should always measure the performance impact of introducing the frozen collections. The number of lookups needed to make these worthwhile will depend on the collection size, and also the key and element types, so there are no fixed rules about

when the frozen collections will offer a benefit, but to give you a rough idea, I ran some tests using dictionaries with 100,000 elements. (You should take this with a grain of salt because all of the factors I just mentioned can make significant differences.) Lookups in this particular test were around 25% faster with the frozen dictionary than with `Dictionary<TKey, TValue>`, but it took over two million lookups to offset the higher initialization costs.

Summary

In this chapter, we saw the intrinsic support for arrays offered by the runtime and also the various collection classes that .NET provides when you need more than a fixed-size list of items. Next, we'll look at a more advanced topic: inheritance.

- 1** Surprisingly, `foreach` doesn't require any particular interface; it will use anything with a `GetEnumerator` method that returns an object providing a `MoveNext` method and a `Current` property. Before generics, this was the only way to enable iteration through collections of value-typed elements without boxing every item. [Chapter 7](#) describes boxing. Even though generics have fixed that, non-interface-based enumeration continues to be useful because it enables collection classes to provide an extra `GetEnumerator` that returns a `struct`, avoiding an additional heap allocation when the `foreach` loop starts. `List<T>` does this.
- 2** Some of this cleanup work happens in the call to `Dispose`. Remember, `IEnumerator<T>` implementations all implement `IDisposable`. The `foreach` keyword calls `Dispose` after iterating through a collection (even if iteration was terminated by an error). If you're not using `foreach` and are performing iteration by hand, it's vitally important to remember to call `Dispose`.
- 3** You might be wondering about `ref readonly`, described in [Chapter 3](#), but that's a different kind of reference. A `ref readonly` to some reference-type variable only means you can't change the variable; it won't stop you making changes to whatever that variable refers to.
- 4** Since end-relative indexes are stored as negative numbers, you might be wondering whether `^0` is even legal, given that the `int` type does not distinguish be-

tween positive and negative zero. It is allowed because, as you'll soon see, `^0` is useful when using ranges, so `Index` is able to make the distinction.

- 5 In cases where you use `^` directly against an `int` inside an array indexer (e.g., `a[^i]` where `i` is an `int`), the compiler generates marginally simpler code. Instead of converting `i` to an `Index`, then calling `GetOffset`, it will generate code equivalent to `a[a.Length - i]`.
- 6 The usual complexity analysis caveats apply—for small collections, the simpler data structure might well win, with the sorted list's theoretical advantage only coming into effect with larger collections.
- 7 There's an exception to this rule: you can use a collection from multiple threads as long as none of the threads attempts to modify it.